okRemote: Design Specification

Oct 31, 2026 · @Taylor

1. Purpose, scope and conventions

This document describes okRemote 0.1.0 (plus the unreleased changes after it) completely enough that a separate team could rebuild the same product from it without seeing the source: what every feature does, how the parts fit, what every menu, dialog and button says and does, the stored formats, the numbers and colors, and the errors users see. It contains behavior and design only, no source code.

1.1 What okRemote is

okRemote is a desktop connection manager for SSH, RDP, VNC, web (HTTP and HTTPS) and arbitrary external-tool sessions. Saved hosts live in a tree of folders inside an encrypted vault file; sessions open as tabs (SSH terminals and RDP desktops) or in their own windows (web, VNC, tools). Folders carry defaults (credentials and options) that everything below inherits. SSH jump chains and forwards are first-class objects. It runs on Windows, macOS and Linux, keeps credentials encrypted at rest, and sends nothing anywhere. Its intended users are people who manage many hosts: homelab owners, system administrators and consultants.

1.2 Baseline and what is not in it

The baseline is a personal, single-user build. These are **not** implemented in this version, and a rebuild should not include them if it wants to match: shared or server workspaces (PostgreSQL), users, groups and grants (their tables exist unused); an SFTP pane; telnet and serial consoles (telnet entries import as external-tool entries); broadcast input; font ligatures; terminal highlight rules (stored, not applied) and background blur; per-connection and per-folder icons (stored, not drawn); detaching a tab into its own window; whole-file encryption of the vault; clipboard, audio, drive and printer redirection, Kerberos and multi-monitor in the embedded RDP client; an in-app restore for backups; a menu item for JSON export (the command exists); and organization key recovery. Installers are unsigned. Performance targets (start time, a 50,000-node tree, terminal throughput) were not measured.

1.3 Terms

Term	Meaning
Workspace	A set of folders, connections, tunnel profiles and terminal profiles. In this version one workspace is one vault file
Vault (file)	The encrypted SQLite file `*.okvault` that stores a workspace
Connection	A saved target: protocol, host, port, options, tags, notes, credentials
Folder	A container with its own defaults (options and credentials)
Option	A named setting on a connection or folder; unset means inherit
Secret	The credentials of a connection or folder: user name, password, domain, private key, key passphrase
Quick target	A connection typed into the top bar and never stored
Tunnel profile	A jump chain of SSH connections plus forwards
Terminal profile	A named terminal appearance spec
Session	One live connection (an SSH shell, an RDP session, an external client)
Tab, pane	A tab shows one SSH or RDP session; a pane holds tabs; panes split
Drawer	The editor panel at the right edge
Prompt	A question from the core to the user (host key, password, credentials)
Route	The tunnel profile label a session runs through

1.4 Conventions in this document

- `Code font` is an exact identifier, option key, setting key, command, file name, event name or on-screen text that must match. Text in double quotes is exact user-visible wording. A key in `<angle brackets>` is a placeholder filled at run time.
- Lengths are CSS pixels and rem units at the 13 px base (section 5.8). Times are UTC unless noted. Colors are CSS hex.
- "Mod" is Ctrl on Windows and Linux and Cmd on macOS.
- "Core" means the Rust side (crates plus the shell); "interface" means the web-view front end.
- A "toast" is a transient message (15.12); a "dialog" is a modal (15.1); a "prompt" is a dialog raised by the core.
- Stored file formats (the vault schema, sealed values, the import formats) are compatibility requirements and are specified exactly. Everything else is behavior to reproduce.

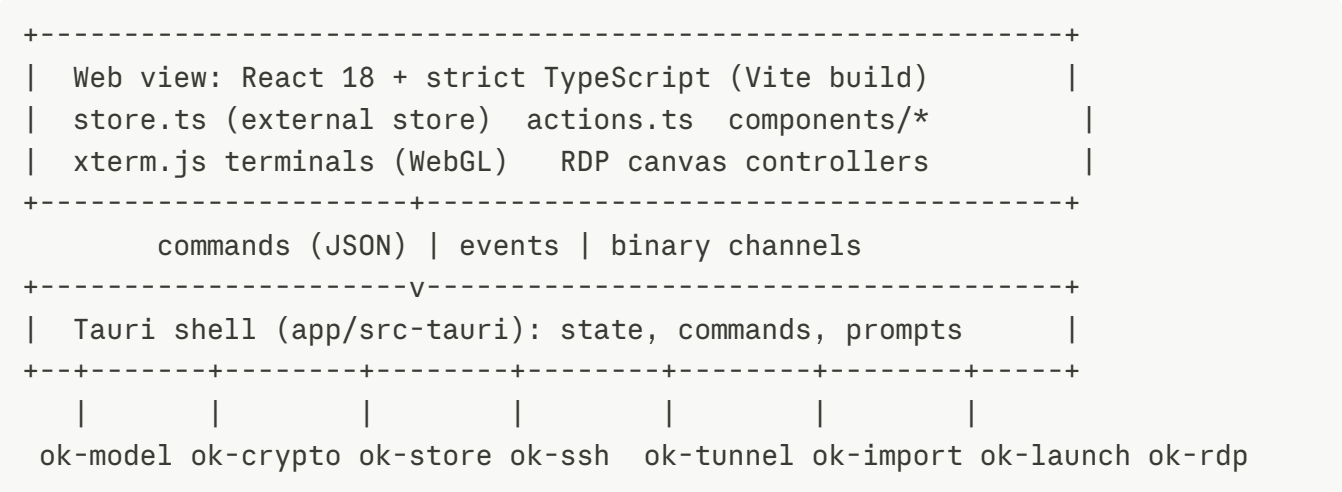
1.5 Map

Section	Covers
2	Architecture, crates, commands and events
3	Data model and option reference
4	Vault file and cryptography
5	Window layout, top bar, status bar, theme
6	Sidebar, filter, tree, drag and drop
7	Connection and folder editor
8	SSH sessions and the terminal
9	RDP
10	VNC, web, external tools
11	Tunnels and jump chains
12	Tabs, panes, welcome view
13	Quick connect, palette, favorites, recents

Section	Covers
14	Import and export
15	Dialogs, menus, toasts
16	Settings and shortcuts
17	Security and audit
18	End-to-end workflows
19	Build, packaging, CI
20	Appendices: commands, errors, color schemes, icons

2. System architecture

okRemote is a desktop shell (Tauri 2) around a Rust core, with a React and TypeScript interface running in the system web view. All secrets, network protocols and file parsing live in Rust; the interface only draws state and sends commands.



2.1 Crates

One Rust workspace, version 0.1.0, edition 2021, minimum Rust 1.85, release profile with thin LTO, 4 codegen units, symbols stripped, panics abort.

Crate	Responsibility	Main libraries
`ok-model`	Shared records (protocol, folder, connection, secret, tunnel profile, terminal profile, host key, audit entry, snapshot), option maps with inheritance and the typed option view, quick-connect parser	serde, uuid (v7), chrono, zeroize
`ok-crypto`	Key type with zeroizing storage, Argon2id derivation with calibration, XChaCha20-Poly1305 sealing with bound associated data, key wrapping, random tokens	argon2, chacha20poly1305, rand
`ok-store`	The `Store` trait (the one interface the app uses) and the SQLite vault implementation: schema, migrations, locking, version-checked saves, host keys, audit, backup	rusqlite (bundled SQLite)
`ok-ssh`	SSH client: connect through a chain of hops, authentication (key, agent, password, keyboard-interactive), host-key callback, shell channel, remote-forward handling, the `Interact` trait through which the core asks the user	russh 0.65 (ring, rsa, flate2)
`ok-tunnel`	Shared transports with health and reconnect, local, SOCKS5 and remote forwards, leases	tokio
`ok-import`	Importers (XML connection tree, OpenSSH, PuTTY, RDCMan, CSV) and color-scheme parsers	roxmltree, csv, aes-gcm, pbkdf2, md-5, winreg (Windows)
`ok-launch`	Launchers for external RDP, VNC and tool clients, with the password channels of 17.3	which, des

Crate	Responsibility	Main libraries
`ok-rdp`	The built-in RDP engine on its own thread: connect, sign in, decode, pump input	IronRDP crates (connector 0.11, session 0.12, graphics, display control, DVC, TLS with rustls)
`app/src-tauri`	The shell: application state, every command, prompts, event emission, window and bundle configuration	tauri 2, dialog, opener and clipboard plugins, keyring

The crates `ok-model`, `ok-crypto` and `ok-store` have no network or interface dependencies. `ok-ssh` and `ok-tunnel` never import the UI; they ask questions through the `Interact` trait.

2.2 Application state (shell)

One shared state object holds: the config paths; the settings JSON (behind a lock); the open workspaces (workspace id to a locked vault); the live SSH sessions (id to input channel, title, workspace, connection, and the leases and transport that keep its route alive); the live embedded RDP sessions (id to input channel); the tunnel manager; the prompt table; the pending-import table (token to parsed result and source); the list of workspace paths requested at launch; and the device host-key list. Every command takes the state, so the same vault lock serializes writes from all callers.

2.3 Commands, events and channels

The interface calls Rust through named commands (full list in Appendix A) that take and return JSON. Failures return an object `{code, message, detail?}` (Appendix B). Long operations such as vault creation run on blocking threads so the window stays responsive.

The core pushes to the interface through events and binary channels:

Mechanism	Name	Payload
Event	`prompt`	A question for the user: an id, a `kind` and kind-specific fields (host key, password, key passphrase, keyboard-interactive prompts, credentials). Answered with the `prompt_respond` command; unanswered for 600 s counts as cancelled
Event	`session-state`	`{id, state: "closed", exit_status}` when an SSH shell ends
Event	`tunnels-changed`	No payload; any change to transports or forwards. The interface refetches the status
Event	`open-workspaces`	List of file paths the OS asked the app to open (macOS file-open events)
Channel	per SSH session	Raw terminal bytes (binary)
Channel	per RDP session	Tagged binary messages (section 9.4)

Prompt payloads: `hostkey_unknown {host, port, key_type, fingerprint, other_keys[], label, shared}`; `hostkey_changed {host, port, key_type, old, new, label}`; `password {user, host, label}`; `key_passphrase {host, key, label}`; `keyboard_interactive {host, name, instructions, prompts[{prompt, echo}], label}`; `credentials {host, name, protocol, need_user, need_password, user, default_user, domain}`. Every prompt also carries its `id` and `kind`.

2.4 Interface structure

The interface keeps one global state object outside React (`store.ts`), read with a selector hook that re-renders a component only when its slice changes. Components: `App` (layout, theme, global shortcuts), `Chrome` (top bar, status bar, first-run), `Sidebar` (filter, favorites, tree), `Editor` (drawer), `Sessions` (panes, tabs, terminal and RDP panes, welcome view), `Dialogs` and its siblings (modals and prompts), `Palette`, `ContextMenu`, plus `ui` helpers (icons, modal frame, field, dot). Terminal and RDP controllers are plain classes kept in registries keyed by tab id, outside React, so a tab moving between panes keeps its live screen. State shape: ready flag, app info, settings, workspaces (info plus

snapshot), tabs, pane tree, active pane, tree selection and focus, filter and chips, expansion map, editor target, open dialog, palette flag, prompt queue, toast list, tunnel status and the first-run flag.

In a plain browser during development (`npm run dev`), a mock backend (`devmock.ts`) answers the commands with sample data so the interface can be worked on without the desktop shell; it is never loaded in a built app.

2.5 Threading

The shell runs commands on the async runtime. SSH, tunnels, forwards and prompts are async tasks. Each embedded RDP session owns an OS thread with a current-thread runtime. Vault calls take the vault lock and are short; creation, unlock and passphrase change (which derive keys) run on blocking threads.

3. Data model

Every stored object is a plain record with a UUIDv7 `id` (time-ordered, safe to generate offline), an integer `sort` for sibling order and an integer `version` used for optimistic concurrency. Credentials are never part of these records; they live in a separate encrypted secret (section 4).

3.1 Protocol

Six protocols, serialized as lowercase strings.

Value	Default port	Opens as
`ssh`	22	Embedded terminal tab
`rdp`	3389	Embedded RDP tab, or a platform client
`vnc`	5900	Platform viewer window
`http`	80	okRemote web window or system browser
`https`	443	okRemote web window or system browser
`external`	0 (none)	A user-defined command line

`Protocol::parse` is case-insensitive and accepts `ssh2` as `ssh`. A connection whose `port` is null uses the protocol default (`effective_port`).

3.2 Folder

Field	Type	Meaning
id	string	UUIDv7
parent_id	string or null	null means top level of the workspace
name	string	Display name
color	string or null	CSS hex such as `#ef4444`; shown as a chip on the tree row and on tabs of descendants
icon	string or null	Stored, not drawn in 0.1.0
defaults	option map	Option values every descendant inherits unless it sets its own
has_secret	bool	True when the folder holds default credentials
sort	integer	Order among siblings
version	integer	0 for a new unsaved record, incremented by every save

3.3 Connection

Field	Type	Meaning
id, folder_id, name, sort, version	as for folder	folder_id null means top level
protocol	Protocol	See 3.1
host	string	Host name or address. For http and https it holds the full URL when the user typed one
port	integer or null	null means the protocol default
options	option map	Only the keys this connection overrides
tags	list of strings	Free text; tags never inherit
notes	string	Free text

Field	Type	Meaning
icon	string or null	Stored, not drawn in 0.1.0
has_secret	bool	True when the connection has its own credentials

3.4 Secret (credentials)

One secret per folder or connection, with five optional string fields: `username`, `password`, `domain`, `private_key` (inline OpenSSH or PEM text) and `key_passphrase`. An empty string counts as unset (`normalized`). Each field inherits independently: a connection that sets only `username` still gets `password` from the nearest ancestor folder that sets one (`inherit_from` fills every unset field from the parent, applied nearest ancestor first). The debug form of a secret redacts password and private key.

3.5 Option maps and inheritance

Options are a flat JSON object. A key that is absent inherits; a key that is present overrides; a JSON null counts as absent. Resolution (`resolve_options`): take the ancestor folders of the connection from root to nearest, then the connection itself, and merge in that order so the leaf wins. Two keys never inherit: `pinned` and `auto_open`. The resolved map is converted to a typed structure leniently: a malformed value falls back to its default and the rest still apply.

Group	Key	Type	Default
SSH	auth_method	auto, password, key, agent, keyboard_interactive	auto (agent, then key, then password, then keyboard-interactive)
SSH	key_file	path	none
SSH	keepalive_seconds	integer	30
SSH	term_type	string	xterm-256color
SSH	terminal_profile	profile id	none (use global terminal settings)
SSH	tunnel_profile	profile id or the word `none`	none

Group	Key	Type	Default
SSH	fixed_cols, fixed_rows	integer	none (auto-fit)
SSH	session_log	bool	false
SSH	session_log_dir	path	none (app log folder)
SSH	session_log_timestamps	bool	false
SSH	osc52	bool	false
SSH	allow_sha1_rsa	bool	false
RDP	rdp_scaling	dynamic, smart, fixed, scroll	dynamic
RDP	rdp_width, rdp_height	integer	none
RDP	rdp_color_depth	8, 15, 16, 24, 32	32
RDP	rdp_scale_factor	integer percent	100
RDP	rdp_clipboard	bool	true
RDP	rdp_drives, rdp_printers	bool	false
RDP	rdp_nla	bool	true
RDP	rdp_admin	bool	false
RDP	rdp_fullscreen	bool	false
RDP	rdp_client_path	path	none (built-in client)
VNC	vnc_view_only, vnc_remote_resize	bool	false
VNC	vnc_client_path	path	none
HTTP	http_open_in	embedded, system	embedded
HTTP	http_zoom	float	1.0
HTTP	http_path	string	none
External	ext_command	string	none
External	ext_password_via	env, stdin, none	env

Group	Key	Type	Default
External	ext_working_dir	path	none
General	pinned	bool	false (never inherits)
General	auto_open	bool	false (never inherits)

3.6 Tunnel profile

`id`, `name`, `chain` (ordered list of SSH connection ids, first hop first; the last hop is where forwards end), `forwards` (list), `auto_start` (start the explicit forwards whenever an attached connection opens) and `version`.

A forward is one of three kinds:

Kind	Fields	Behavior
local (-L)	bind_addr, bind_port, remote_host, remote_port	Listen locally, connect from the last hop to remote_host:remote_port
dynamic (-D)	bind_addr, bind_port	Local SOCKS5 proxy through the last hop
remote (-R)	remote_bind_addr, remote_bind_port, local_host, local_port	Listen on the SSH server, connect back to local_host:local_port

A forward "exposes to network" when its bind address is anything other than `127.0.0.1`, `::1`, `localhost` or empty; those need user confirmation (section 11).

3.7 Terminal profile

`id`, `name`, `spec` (a JSON object whose shape is owned by the interface, section 8.4) and `version`. A profile can be set globally, on a folder or on a connection through the `terminal_profile` option.

3.8 Host key and audit entry

Record	Fields
Host key	host, port, key_type, fingerprint, first_seen (RFC 3339), trusted_by
Audit entry	id, at (RFC 3339 UTC, seconds), actor, action, target_id, detail (JSON)

3.9 Snapshot

The interface receives a workspace as a snapshot of its non-secret content: folders, connections, tunnel profiles and terminal profiles. Ancestors of a folder are walked nearest first with a guard of 4,096 steps against cycles.

3.10 Quick-connect target

Parsing rules for the top-bar field, also used by the command palette:

1. Trim; empty input is an error ("Enter a host to connect to").
2. A `scheme://` prefix selects the protocol (ssh, ssh2, rdp, vnc, http, https). Any other scheme, including `external`, is an error. No scheme means ssh.
3. The authority ends at the first `/`, `?` or `#`. For ssh, rdp and vnc any path other than empty or `/` is an error.
4. `user@host` splits at the last `@`. An inline `user:password` keeps only the user; the password is dropped and never stored. The user is percent-decoded.
5. IPv6: `[addr]:port` is accepted; an unbracketed address with more than one colon is taken whole as the host with no port.
6. A port must parse as 1 to 65535; `0` and out-of-range values are errors.
7. For http and https the result also carries the full URL (`scheme://hostport` plus path and query).
8. The display name is `user@host:port`, omitting parts that are absent.

4. Vault file and cryptography

A workspace is one SQLite file with the extension `.okvault`. Only the credential fields are encrypted; folder names, host names, notes and tags are plain text in the file. A clean-room build must keep the formats below exactly if it wants to open files made by this one.

4.1 Key hierarchy

1. The user passphrase goes through Argon2id (version 0x13) with the stored parameters and a 16-byte random salt, and yields a 32-byte key-encryption key (KEK).
2. A random 256-bit vault key is generated once per vault. It is stored only as a wrapped value: the vault key sealed with XChaCha20-Poly1305 under the KEK.
3. Each folder or connection secret is sealed under the vault key.
4. All key material is held in zeroizing buffers and cleared when dropped. The vault key is held in memory only while the workspace is unlocked.

Argon2id parameters are stored as JSON in `workspace.kdf_params`: `{"alg":"argon2id","m_cost":65536,"t_cost":N,"p_cost":4,"salt":"<base64>"}`. `m_cost` is in KiB (64 MiB). Default `t_cost` is 3. On vault creation and passphrase change the iteration count is calibrated: derive one key with t_cost 3, time it, scale `t_cost` so one derivation takes about 300 ms on this machine, and clamp to 3 through 12. Any algorithm name other than `argon2id` is refused.

4.2 Sealed value format

`seal(key, plaintext, aad)` returns the string `xc20p:v1:` followed by the base64 (standard alphabet, padded) of `nonce || ciphertext`, where the nonce is 24 random bytes and the ciphertext includes the 16-byte Poly1305 tag. Opening rejects a missing prefix, bad base64 and anything shorter than 24 + 16 bytes as a format error, and an authentication failure as "wrong passphrase or corrupted data".

Associated data binds a ciphertext to its place: `aad(owner_id, field)` is the owner id bytes, one 0x00 byte, then the field name bytes. Ciphertext copied into another row or another field therefore fails to open.

Purpose	owner_id	field
Wrapped vault key	workspace id	`wrapped_key`
Credential secret (folder or connection)	that record's id	`secret`

Purpose	owner_id	field
Canary	workspace id	`canary`

A secret is serialized as JSON containing only its set fields (`username` , `password` , `domain` , `private_key` , `key_passphrase`), then sealed. An empty secret is stored as null, which clears it.

The canary is the fixed bytes `okremote-vault` sealed under the vault key and stored in `settings` under the key `canary` . It lets a vault key released by the OS keychain be checked without the passphrase.

4.3 SQLite schema (version 1)

Pragmas: `application_id = 0x4F4B524D` ("OKRM"), `user_version = 1` , `journal_mode = WAL` , `foreign_keys = ON` , `synchronous = NORMAL` , busy timeout 5 seconds.

Table	Columns
workspace	id (pk), name, schema_version, kdf_params (JSON), wrapped_key, created_at
folders	id (pk), parent_id (fk folders, cascade delete), name, color, icon, defaults (JSON, default `{}`), secret_blob, sort (default 0), version (default 1); index on parent_id
connections	id (pk), folder_id (fk folders, cascade delete), name, protocol, host, port, options (JSON), tags (JSON array), notes, icon, secret_blob, sort, version; index on folder_id
tunnels	id (pk), name, kind, spec (JSON holding forwards and auto_start), chain (JSON array of connection ids), version
terminal_profiles	id (pk), name, spec (JSON), version
host_keys	host, port, key_type, fingerprint, first_seen, trusted_by; primary key (host, port, key_type)

Table	Columns
audit_log	id (pk), at, actor, action, target_id, detail (JSON); index on at
settings	key (pk), value
users, groups, group_members, folder_keys, grants	Created but unused in 0.1.0; reserved for team sharing

Deleting a folder deletes its subfolders and connections through the cascade. `has_secret` on a record is derived from whether `secret_blob` is non-null.

4.4 Lifecycle

- **Create.** Refuses an existing path and an empty passphrase. Writes schema, workspace row, canary and pragmas in one transaction, records `workspace.create`, then the app adds one folder named `Example` and (optionally) caches the key in the OS keychain.
- **Open.** The file must be a SQLite database whose `application_id` matches, else "not an okRemote vault file". A `user_version` above the build's is opened read-only (all writes fail with "the workspace is open read-only" and audit recording is skipped). A lower version is migrated forward inside one transaction, after copying the file to `<name>.pre-migration-YYYYMMDD-HHMMSS.okvault` beside it.
- **Unlock.** By passphrase (derive, unwrap; failure is "wrong passphrase"), or by a cached key (canary check). A workspace can be open and locked: the tree is hidden and every secret or write operation fails with "the workspace is locked".
- **Lock.** Drops the vault key. Existing sessions and tunnels keep running.
- **Change passphrase.** Verifies the old passphrase, derives a new calibrated KEK, rewraps the same vault key. No secret is re-encrypted. Records `workspace.passphrase`.
- **Rename.** Updates the workspace row name.
- **Backup.** `VACUUM INTO` a new file, safe while open. See 4.6.

4.5 Saves and conflicts

A record with `version` 0 is inserted with version 1. Otherwise the update runs only when the stored version equals the sent version, and then increments it. If no row matched, the save fails with a conflict that carries the record as it now stands (or not-found if it was

deleted). The interface uses that to offer keep mine, keep theirs or a field-by-field merge (section 7.6). Empty names are refused ("folder name is required", "connection name is required"). A folder cannot be its own parent or moved under one of its own descendants.

4.6 Backups and device cache

- A backup is written every time a vault is opened unlocked or unlocked, to `<config>/backups/<workspace id>/YYYYMMDD-HHMMSS.okvault` (UTC). The newest `backup_retention` files are kept (setting, default 10; 0 turns backups off); older ones are deleted by sorted file name. A manual "back up now" runs the same routine.
- "Remember on this device" stores the base64 vault key in the OS keychain: service `OkRemote`, account `vault:<workspace id>` (macOS Keychain, Windows Credential Manager, Linux Secret Service). On open without a passphrase the app tries that key; if the canary check fails the entry is deleted. Changing the passphrase re-stores the key when one was cached. There is no passphrase recovery.

4.7 Files outside the vault

File	Location	Content
settings.json	config folder	Interface and app settings (section 16), recent and last-open workspace paths (12 each, newest first)
known_hosts.json	config folder	Host keys trusted for quick-connect sessions (workspace connections store theirs in the vault)
backups/	config folder	Vault backups
logs/	config folder	Default folder for SSH session logs (8.7); the app itself writes no log file

The config folder is `<OS config dir>/OkRemote`. Portable mode: if a file named `portable` or `okremote.portable` sits next to the executable, the config folder is `data/` beside the executable. The names `OkRemote` (folder, default vault folder, keychain service) and the `okvault` extension are storage identifiers; the window title and visible product name are `okRemote`. The default vault path offered to the user is `<Documents>/OkRemote/personal.okvault`.

Workspace paths given on the command line open at startup: `--workspace PATH`, `-w PATH`, `--workspace=PATH` (repeatable), or a bare argument ending in `.okvault` (case-insensitive). On macOS a file-open event from the OS is queued the same way and also emitted to the interface as `open-workspaces`.

5. Window layout, theme and dimensions

5.1 Window

One main window titled `okRemote`, opening at 1400 by 900 pixels, minimum 820 by 520, with native file drops disabled (so a dropped file never navigates the web view). The browser's own context menu is suppressed everywhere except inside text inputs; right-click opens the application's own menus instead. Before the interface has loaded the window shows the text "Starting…".

A content security policy limits the page to its own scripts and the application bridge: `default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; font-src 'self' data:; connect-src 'self' ipc: http://ipc.localhost`. Windows opened for web connections load remote pages and get no bridge permissions at all; the main window may set its title, close itself, show open and save dialogs, ask and confirm and message dialogs, open URLs, reveal files in the file manager and read or write the clipboard as text.

5.2 Regions

The page is a vertical stack filling the window:

1. **Top bar** (padding 6 px by 10 px, 8 px gap, bottom border).
2. **Main area** (fills the rest), a horizontal row of: the sidebar (unless collapsed), a 4-pixel resize handle, the session area (or the first-run screen), and the editor drawer, which overlays the right edge.
3. **Status bar** (padding 3 px by 10 px, 16 px gap, text 0.82 rem, top border).

Overlays above everything, in this order from bottom to top: dialogs, prompts, command palette, context menu, toasts.

5.3 Top bar

Left to right:

Control	Behavior
Sidebar icon button	Toggles the sidebar. Tooltip "Toggle sidebar (Ctrl+B)". Label changes between "Show sidebar" and "Hide sidebar"
Quick-connect field	A bordered rounded box (stretches, max 720 px wide) holding a terminal icon, a text input with the placeholder `Quick connect: user@host:port, rdp://host, https://…`, a small primary **Connect** button and a small **Save** button. Both buttons are disabled while the input is blank. Enter submits as Connect. Escape blurs the input
Palette button	Small button showing a search icon and the key hint `Ctrl K`; opens the command palette
Lock button	Lock icon. Disabled when no workspace is unlocked. Locks every workspace
Menu button	Opens the application menu (5.4) just below the button

The application name is not shown in the top bar. Connect parses the text (section 3.10), clears the field and opens the connection as a quick target. Save creates a connection from the text instead: name and host from the target (for web targets the host is the full URL and the port is left empty), placed in the insertion point of section 6.9; if the text carried a user name it is stored as that connection's credential; then the editor drawer opens on the new connection. With no unlocked workspace, Save shows the error "Open a workspace to save connections".

5.4 Application menu

A context menu with these items, in order: **Open or create a vault…**, **Import…**, separator, **Tunnels…**, **Terminal profiles…**, **Host keys…**, **Audit log…**, separator, **Settings…**, **About okRemote**. Each opens the dialog of that name (section 15).

5.5 Sidebar region

Width is the setting `sidebar_width` (default 290 px, dragged between 200 and 600 px, saved), fixed (no flex shrink). A 4-pixel vertical handle on its right edge resizes it. When `sidebar_collapsed` is true neither the sidebar nor the handle is drawn. Contents are described in section 6.

5.6 Session area

Fills the remaining width. It holds a tree of panes (section 12). When no workspace exists, a first-run screen replaces it (section 18.1). The editor drawer is positioned absolutely against the main area: right edge, full height, width `min(480px, 92vw)`, left border and shadow, slide-in animation of 140 ms, stacked above the session area.

5.7 Status bar

Left: a vault icon (a lock icon when every workspace is locked) and `N workspace(s)` plus `, M locked` when any are locked; hovering lists each workspace and its state. Next: a button with a tunnel icon, a health dot (shown only when a transport is failed or reconnecting, or a forward is running) and `N forward(s)`, plus `· <transport label> <state>` for the worst transport; click opens the Tunnels dialog. A spacer. Right: for the active tab, `<PROTOCOL> · Fixed CxR` or `Auto-fit CxR`, then `· zoom +N` when zoomed, `· via <route>` when tunneled, `· logging` when a session log is being written. For an RDP tab the columns and rows are 0 until measured.

5.8 Theme

The root element carries `data-theme` (`light` or `dark`; the setting `system` follows the operating system's preference live), `data-density` (`comfortable` or `compact`) and CSS variables. Root font size is `13 px × ui_scale`, with the scale clamped to 0.8 through 2; every length in the stylesheet is in `rem` or pixel values chosen against that base.

Variable	Light	Dark
--bg (page)	#f5f4f2	#1e1e1e
--bg-2 (panels, top bar, sidebar)	#ffffff	#262626
--bg-3 (tab strip, chips)	#ebe9e6	#2e2e2e
--bg-hover	#e1dfdb	#383838

Variable	Light	Dark
--border	#d6d3ce	#3b3b3b
--text	#1f1e1d	#ececec
--text-2	#55524e	#b4b4b4
--text-3	#78746e	#8a8a8a
--danger	#c62828	#f2777f
--warn	#9a6700	#e8c46a
--ok	#1f7a35	#7fd18f
--sel (selection fill)	accent at 18%	accent at 28%
--shadow	0 8px 30px rgba(0,0,0,.16)	0 8px 30px rgba(0,0,0,.5)
--term-bg	#faf9f7	#181818

Fixed variables: accent `#9fcdba` (user-settable; the text color on accent surfaces is `#1d2a25` when the accent's luminance `0.299R + 0.587G + 0.114B` over 255 exceeds 0.6, else `#ffffff`), radius 6 px, tree row height 28 px (22 px compact), transition 120 ms. Fonts: UI stack `-apple-system, BlinkMacSystemFont, Segoe UI, system-ui, Ubuntu, Noto Sans, sans-serif`; monospace stack `ui-monospace, Cascadia Mono, Menlo, Consolas, DejaVu Sans Mono, monospace`. Text is not selectable except in inputs, toasts and read-only code fields.

Controls: buttons are bordered, 5 px by 12 px padding, radius 6, secondary buttons use `--bg-2`; **primary** buttons use the accent with `--on-accent` text; **danger** buttons use the danger color; **small** buttons are tighter. Icon buttons are 28 by 28 px, borderless, `--text-2`. Focus shows a 2 px accent outline. Chips are small rounded toggles; a chip that is on is filled with the accent. A badge is a 0.72 rem pill with a border. Connection status dots are 8 px circles: connected green (`--ok`), connecting amber, degraded amber, error red, closed and idle grey. Selects are drawn with a custom chevron so macOS shows no native gradient.

6. Sidebar and tree

The sidebar is a vertical stack: the filter box, an optional chip row, the favorites strip (only when there are favorites), the tree (fills the remaining height and scrolls) and a row of three action buttons.

6.1 Filter box and chips

- A bordered box with a search icon, a text input (placeholder `Filter connections`) and a menu icon button labeled "Filters". The button is highlighted while the chip row is open or any filter is active; clicking it toggles the chip row.
- Escape in the input clears the text and blurs it. Arrow Down moves keyboard focus into the tree.
- The chip row (hidden until opened) holds toggle chips in this order: protocol chips (SSH, RDP, VNC, HTTPS, HTTP, External tool), tag chips (`#tag`, up to 40, alphabetical, from every unlocked workspace), workspace chips (only when more than one workspace is open) and a **Clear** chip when any filter is active. Chips use `aria-pressed`.
- The tree is in **search mode** whenever the text is non-blank or any chip is on.

6.2 Search semantics

Split the text on whitespace into lowercase terms. A connection matches when every term appears somewhere in the text `name \n host \n tags \n notes \n folder path` (lowercased; the path is folder names joined by `/`). Chips: protocol chips match if the connection's protocol is any selected one; tag chips require every selected tag; workspace chips match if the workspace is any selected one. Locked workspaces are skipped. At most 500 results. Results whose name starts with the first term come first; otherwise the order is workspace order then tree order. Only connections are listed, flat, with no expander, no indentation and no workspace rows. Each row's right-hand text becomes `<folder path> · <host>`.

6.3 Favorites strip

Shown when any connection in any unlocked workspace has `pinned` true. A horizontally scrolling row of small bordered buttons (max 160 px each) with the protocol icon (14 px) and the name; tooltip `Connect to <name>`; click opens the connection.

6.4 Rows

The tree is virtualized: rows are absolutely positioned at `index × row height` (28 px, 22 px compact) inside a container of full height; only rows within the viewport plus 12 rows of overscan are rendered. Row `padding-left` is `8 px + depth × 6 px` in normal mode and 8 px in search mode. Depth is 0 for a workspace, 1 for its top-level items, and so on.

Every non-search row begins with a 14 px **twisty** (a chevron icon, rotated 90° when open; invisible but still taking space on leaves so names align). Then:

Row	Contents after the twisty
Workspace	Vault icon (lock icon when locked), bold name, a `read-only` badge when the vault is read-only, and an **Unlock** link button when locked
Folder	Folder icon (open-folder icon when expanded), a small color chip when the folder or an ancestor has a color, the name
Connection	Protocol icon tinted by protocol, a small color chip (inherited folder color), the name (max 70% width, ellipsis), a small star when pinned, and the host right-aligned in smaller, dimmer text that takes the remaining space

Order within a parent: folders first, then connections; each sorted by `sort` then by name (locale-aware). Workspace roots are ordered with the pinned workspace first (the one whose file path equals the `pinned_workspace` setting), then by name.

A workspace is expanded unless a `<row key>:closed` marker is in the expansion map. A folder is expanded when its key is in the map. Row keys are `workspace:kind:id`. The map is saved in the `expanded` setting (debounced 400 ms). A folder with no children has no expander.

6.5 Selection and focus

- Click selects the row alone; Ctrl or Cmd click toggles it; Shift click selects the range between the anchor row and the clicked row, limited to non-workspace rows of the same workspace.
- Clicking a workspace row clears the selection and focus.

- The selected style fills the row with the selection color; the focused row also shows a focus outline while the tree has keyboard focus. Focusing the tree with nothing focused selects the first non-workspace row.
- Double-click opens a connection, or toggles a folder or workspace (a locked workspace opens the unlock dialog instead).
- Clicking a twisty toggles without selecting.

6.6 Keyboard in the tree

Key	Action
Down, Up	Move focus; with Shift, extend the selection. Up from the first row in search mode returns to the filter input
Right	Expand a collapsed folder, otherwise move down
Left	Collapse an expanded folder
Home, End	First, last row
Enter	Connect (connection) or toggle (folder, workspace)
F2	Open the editor on the focused item
Delete	Delete the selection (after confirmation)
Alt+Up, Alt+Down	Move the focused item one place among its siblings

6.7 Drag and drop

Rows other than workspaces are draggable. The payload (type `application/x-okremote-items`) is the JSON list of selected items, or the dragged row alone if it is not selected. While dragging, a drop indicator class is set on the target row by the pointer's vertical position:

Target	Top part	Middle	Bottom part
Workspace	into	into	into

Target	Top part	Middle	Bottom part
Folder	before (top 25%)	into	after (bottom 25%)
Connection	before (top 50%)		after (bottom 50%)

On drop: items from a different workspace are refused with the toast "Moving between workspaces is not supported yet; export and import instead". A folder cannot be dropped into itself or its descendants ("A folder cannot be moved into itself"). Holding Ctrl or Alt makes the drop a copy; otherwise a move. The new parent is the target folder for "into", the target's own parent for before and after, or the workspace top level. Folders and connections are ordered separately: a "before" target applies only to items of its own kind (an "after" drop is turned into "before the next sibling of the same kind"). Moves renumber the `sort` of every sibling in the destination so the order is stable and only save rows whose sort or parent changed. After a move or copy the destination folder is expanded.

6.8 Context menus

Right-clicking an unselected row first selects it. Menu items:

Row	Items
Workspace	Unlock… or Lock; New connection; New folder; separator; Tunnel profiles…; Terminal profiles…; Import into this workspace…; Audit log…; separator; Close workspace. All but Lock/Unlock and Close are disabled while locked
Connection	Connect (hint Enter; `Connect to N` for a multi-selection); Pin to favorites or Unpin from favorites; separator; Edit (F2); Duplicate; `Bulk edit N connections…` (only when several connections are selected); Move up (Alt+Up); Move down (Alt+Down); separator; Copy user name; Copy password; Copy host; separator; Delete (Del, danger; `Delete N items` for several)

Row	Items
Folder	New connection here; New subfolder; Connect to all in folder; separator; then Edit, Duplicate, Bulk edit, Move up, Move down, separator, Delete as above

"Connect to all in folder" opens every connection directly inside that folder (not subfolders) and is refused above 20 ("That folder has more than 20 connections; select the ones you need"). Copy user name and password go through the credential copy path (section 17.2) and respect inheritance. Pin toggles the `pinned` option on the connection.

6.9 Creating, duplicating and deleting

- **Insertion point** for new items: the focused folder, or the folder of the focused connection, or the top level of the first unlocked workspace. With no unlocked workspace: "Open or unlock a workspace first".
- **New folder** asks for a name in a small dialog (title "New folder", label "Folder name", default "New folder"), creates it with sort 9999, expands the parent, and focuses and selects the new folder.
- **New connection** opens the editor drawer on a blank record (section 7).
- **Duplicate** copies the record with the name suffix `(copy)` in the same folder, drops the `pinned` and `auto_open` options, and re-seals the credentials under the new id. Duplicating a folder copies its defaults, credentials, and every connection and subfolder inside it recursively (only the top copy gets the suffix).
- **Delete** asks "Delete item?" (or `Delete N items?`) listing up to 8 names, and adds "Deleting a folder deletes everything inside it." when a folder is included. Items already gone are ignored silently. The selection, focus and open editor are cleared.

6.10 Empty states and action row

Empty tree text: "No connections match." (search), "No workspace is open." (none open), "Empty. Press Ctrl+N to add a connection." (workspace with no items). The bottom row has three small buttons: **+ Connection** (new connection), **Folder** (new folder), **Vault** (opens the vault dialog).

7. Connection and folder editor

The editor is a drawer on the right edge of the main area (section 5.6). One drawer edits either a connection or a folder; for a folder the same option fields edit its **defaults**, so everything below that folder inherits them.

7.1 Opening and closing

- Opened by: F2 on a focused row (double-click connects instead), the **Edit** menu item, the tab menu's **Edit connection**, **New connection** (a blank record), the quick-connect **Save** button, and nothing else; creating a folder only selects it.
- A blank connection is: new UUID, the chosen folder, empty name and host, protocol `ssh`, no port, no options, `sort` 9999, `version` 0.
- The drawer is keyed by workspace and record, so switching to another record resets it. It shows nothing if the workspace is locked or its snapshot is missing.
- Header: the title (`New connection`, or the name, or `Untitled`), and under it the folder path (or "Top level") plus `· read-only` for a read-only vault; a close icon button.
- Escape or the close button closes it. If the draft differs from the saved record or any credential edit is pending, a confirm dialog asks "Discard changes?" / "Your edits have not been saved." with a danger button **Discard**. Ctrl or Cmd+S saves.
- The drawer is a tab list of sections: **General**, **Credentials**, **Tunnel**, **Display**, **Terminal**, **Advanced**. A section is listed only if it has content for the current protocol (General and Credentials always). For a folder every protocol's fields count, so every section is listed.

7.2 General section

Connection:

Field	Control and rules
Name	Text, autofocused; required
Protocol	Select: SSH, RDP, VNC, HTTP, HTTPS, External tool. Changing it changes which fields appear; option values already entered are kept

Field	Control and rules
Host (label "Host or URL" for HTTP and HTTPS)	Text; placeholder `10.0.0.5 or host.example.com`, or `intranet.example.com or https://…` for web; required unless the protocol is External
Port	Number 1 to 65535; the placeholder is the protocol default (22, 3389, 5900, 80, 443, none); empty means default
Tags	Text, comma-separated; trimmed, empty entries dropped
Notes	4-row monospace text area
Pin to favorites	Checkbox, stores `pinned` (never inherited)
Open automatically when the workspace unlocks	Checkbox, stores `auto_open` (never inherited)

Folder: **Folder name** (required), and **Color** with a row of swatches: a "none" swatch, then `#ef4444`, `#f97316`, `#eab308`, `#22c55e`, `#14b8a6`, `#3b82f6`, `#8b5cf6`, `#ec4899`, `#64748b`, and a free color picker (starting value `#3b82f6`). The hint is "Shown on the tabs of sessions from this folder". A note says settings on the other sections become defaults for everything in the folder.

7.3 Credentials section

Rows shown, in order: **User name**, **Password**, **Domain** (RDP and External connections, and folders), **Private key (inline, encrypted in the vault)** as a 4-row multi-line field (SSH and folders), **Key passphrase** (SSH and folders). After them come the option fields of this section (SSH: **Authentication**, **Private key file**).

Each credential row shows where the value comes from, using the summary the core computes without revealing secrets (own, or the name of the nearest ancestor folder that sets it; user name and domain also show their value). States:

- Own value: the input shows the stored user name or domain text; for secrets it is empty with the placeholder `•••••••• (stored)`.
- Inherited: a `↳ <folder>` mark next to the label, placeholder `Inherited from <folder>` plus `: <value>` for user name and domain.

- Not set: placeholder `Not set`.
- Cleared by the user: placeholder `Inherits from the folder`.

Buttons on a row: an eye (secrets only, not on a new record, when a value exists) with the tooltip "Reveal (recorded in the audit log)"; a copy icon for user name and password (not on a new record); a reset icon "Clear, and inherit from the folder" when the row has an own value or a pending edit; and an **Undo** link while an edit is pending. Typing sets a pending edit; reset sets a pending clear. Nothing is sent until Save. Save sends a patch: a string sets the field, null clears it, fields not touched are unchanged.

A note at the bottom reads "Empty fields inherit from the folder. Values are encrypted in the vault."

7.4 Option fields

Every option field shows its label, the control, and an inheritance mark when it is not set on this record: `↳ default` or `↳ from <folder>`. A set field shows a small reset icon "Reset to the inherited value" that removes the key from this record's options. Writing `undefined` deletes the key; an emptied text or number input deletes it; select values keep their type (numbers stay numbers). Unset text inputs show the inherited value as placeholder.

Section	Field (key)	Control	Shown for
Credentials	Authentication (`auth_method`)	Select: Automatic (key, agent, password); Password; Private key; SSH agent; Keyboard-interactive	SSH
Credentials	Private key file (`key_file`)	Text, placeholder `~/.ssh/id_ed25519`	SSH

Section	Field (key)	Control	Shown for
Tunnel	Tunnel profile (`tunnel_profile`)	Select: Inherit (shows the inherited profile name), None (connect directly), then each profile. Hint: the connection routes through this jump chain; other protocols get a local forward on an ephemeral port. A button **Manage tunnel profiles** opens the Tunnels dialog	All
Display	Scaling (`rdp_scaling`)	Select: Dynamic resolution; Smart size (scale to fit); Fixed resolution; Fixed, with scroll	RDP
Display	Width, Height	Number	RDP
Display	Desktop scale (%)	Select: 100, 125, 150, 175, 200, 250, 300, 400, 500	RDP
Display	Color depth	Select: 32, 24, 16, 15, 8 bit	RDP
Display	Full screen; Clipboard redirection; Drive redirection; Printer redirection	Checkboxes	RDP
Display	View only; Ask the server to resize	Checkboxes	VNC
Display	Open in (`http_open_in`)	Select: okRemote window; System browser. Hint: tunneled web connections always use an okRemote window	HTTP, HTTPS
Display	Zoom	Select: 75%, 90%, 100%, 110%, 125%, 150%, 200%	HTTP, HTTPS
Display	Path	Text, placeholder `/admin`	HTTP, HTTPS

Section	Field (key)	Control	Shown for
Terminal	Terminal profile	Select: Inherit (shows inherited profile name or "app default"), then each profile; button **Manage terminal profiles**	SSH
Terminal	TERM	Text	SSH
Terminal	Fixed columns, Fixed rows	Number; hint "Leave empty to fit the pane."	SSH
Terminal	Log session to a file	Checkbox; warning "The log can contain secrets typed into the session, such as passwords at sudo prompts."	SSH
Terminal	Log folder	Text, placeholder "Default: the app's logs folder"	SSH
Terminal	Timestamp each line	Checkbox	SSH
Terminal	Allow remote programs to set the clipboard (OSC 52)	Checkbox	SSH
Advanced	Keepalive (seconds, 0 = off)	Number	SSH
Advanced	Allow ssh-rsa (SHA-1) host keys	Checkbox; warning "SHA-1 signatures are weak. Enable only for old servers."	SSH
Advanced	Network Level Authentication; Admin / console session	Checkboxes	RDP
Advanced	RDP client (`rdp_client_path`)	Text, placeholder "Default: mstsc / FreeRDP / Windows App"	RDP
Advanced	VNC viewer	Text, placeholder "Default: TigerVNC or Screen Sharing"	VNC

Section	Field (key)	Control	Shown for
Advanced	Command line (`ext_command`)	Text, placeholder `putty.exe -telnet %HOST% %PORT%` ; hint lists the variables and the password channels	External
Advanced	Pass the password through	Select: Environment variable; Standard input; Don't pass it	External
Advanced	Working folder	Text	External

A checkbox field that is on and has a warning shows the warning under it.

7.5 Footer and saving

Buttons: **Delete** (left, danger, only for saved records; runs the delete flow of section 6.9), a spacer, **Cancel**, **Save and connect** (connections only) and **Save** (primary). Save is disabled while saving, for a read-only vault, and when nothing changed on an existing record. Save and connect is disabled while saving or read-only.

Order of work: validate ("A name is required"; "A host is required" unless the protocol is External); trim name and host; save the record; apply any credential patch; refresh the workspace; toast `Saved <name>` for 2.5 seconds; close the drawer and select the saved item; for Save and connect, open the connection.

7.6 Conflict resolution

If the core reports that the record changed since it was loaded (section 4.5), a dialog titled "This item was changed elsewhere" lists every field that differs, except `id` , `version` and `has_secret` , as table rows with two radio choices (Mine, Theirs, default Mine) and the values as JSON or text. Footer buttons: **Keep theirs** (cancel the save and reload), **Keep mine** (resend my record with their version number) and **Save merged** (their record with my chosen fields, their version number). The chosen result is saved again; a second conflict repeats the dialog.

8. SSH sessions and the terminal

8.1 Opening a session

1. The interface creates a tab (id `tab-N`, title = connection name, protocol ssh, state `connecting`, color = nearest folder color) in the active pane (or the pane given), saves the list of open tabs to settings, and, on the next animation frame so the terminal has its real size, starts the session.
2. The core resolves the connection (section 3): options after folder inheritance, credentials merged per field, tunnel profile. A non-SSH connection is refused ("not an SSH connection").
3. If the user name is blank, a credentials dialog asks for it (8.2). Cancelling ends with "<name> was not opened: no credentials were entered", the tab closes and an info toast shows the message.
4. The route is chosen (section 11.2): direct, or through the shared transport of the tunnel profile. The tab later shows the route label.
5. The terminal size is `fixed_cols` by `fixed_rows` when both are set (and the grid is then locked), otherwise the measured size of the pane.
6. If `session_log` is on, a log file is created (8.7).
7. A shell channel opens with a pseudo-terminal of type `term_type` (default `xterm-256color`), columns and rows at least 1, pixel sizes 0, then a shell. Output bytes stream to the interface as raw binary over a channel; bytes arriving before the open call returns are buffered and written first.
8. On success the tab becomes `connected`; the terminal sends its current size to the server and takes focus. On failure the tab becomes `error` and a toast shows `<title>: <message>` for 9 seconds; if sign-in was cancelled the tab becomes `closed` with the message "Sign-in was cancelled.".

When the remote side closes the channel the core emits `session-state` with the exit status; the tab becomes `closed`, the terminal prints a dim line `[session closed]` and the overlay in 8.8 appears.

8.2 Credential prompts

All prompts are queued in the order they arrive and shown one at a time as modal dialogs. A prompt that nobody answers for 600 seconds counts as cancelled.

Prompt	When	Dialog
Sign in / User name	SSH with no user name; RDP with a blank user name or password; VNC with a blank password. For this launch only, nothing is saved	Title "Sign in" (or "User name" when only the user is missing). Text: "No saved credentials for <name> (<host>). They are used for this connection only and are not saved." Fields: **User name** (SSH default is the operating system user), **Domain (optional)** for RDP, **Password** (hidden). **Connect** is disabled until the required fields are filled; **Cancel** aborts the launch
Password	`auth_method = password` with no stored password	Title "Password", subtitle `user@host · <label>`, one hidden field, buttons Cancel and OK
Private key passphrase	The key is encrypted and no passphrase is stored	Title "Private key passphrase", subtitle `Key: <file or "stored key">`, one hidden field
Keyboard-interactive	The server sends prompts	Title = the server's prompt name or "Authentication", subtitle `host · <label>`, the server's instructions as preformatted text, one field per prompt (visible text when the server asks for echo, hidden otherwise), buttons Cancel and Continue

8.3 Authentication

Per hop, with the connection's user name (default: the operating system user), `auth_method` and available secrets:

- **Automatic.** Try in order: the private key (only when an inline key or a key file is configured), the SSH agent, the stored password (never prompts in this mode), then keyboard-interactive. The first success wins; if the server closed the connection the loop stops. If nothing works: "authentication failed for user@host:port".
- **Password, Key, Agent, Keyboard-interactive.** Only that method is tried.

- **Key.** The inline key is parsed first; otherwise the file (a leading `~` expands to the home folder). An encrypted key with no stored passphrase triggers the passphrase prompt (cancel aborts with "cancelled"). RSA keys use the best signature hash the server supports.
- **Agent.** On Unix, the agent named by `SSH_AUTH_SOCK`. On Windows, the OpenSSH named pipe `\\.\pipe\openssh-ssh-agent`, then Pageant. Each identity the agent offers is tried until one succeeds.
- **Keyboard-interactive.** Up to 10 rounds. When a round has exactly one hidden prompt and a password is known (stored, or entered earlier in this attempt) it is used once, silently. Otherwise the keyboard-interactive dialog shows.
- **Host key algorithms.** `ssh-rsa` with SHA-1 is removed from the allowed list unless `allow_sha1_rsa` is set.

8.4 Host key trust

Every hop's server key is checked with the key type and a SHA-256 fingerprint in OpenSSH form (`SHA256:` plus base64). The trust store is the workspace's host-key table for connections that belong to a workspace, or the device file `known_hosts.json` for quick-connect sessions. Matching is by host name (case-insensitive), port and key type.

- **Known and equal:** continue silently.
- **Known but different:** show the blocking dialog **Host key changed: connection blocked** (red warning text that someone could be intercepting the connection or the server was reinstalled; read-only fields **Trusted fingerprint** and **Fingerprint offered now**; the advice to remove the old key under Host keys; buttons **Close** and **Open Host keys**). The connection fails with "host key for host:port was not trusted".
- **Unknown:** dialog **Unknown host key** (wide): "This is the first connection to <host>:<port> (<label>). Check the fingerprint with the server's administrator before trusting it.", the fingerprint in a monospace field labeled with the key type, a warning when the host is known with other key types, and a note saying a trusted key is saved in the workspace (shared) or on this device. Buttons **Cancel**, **Connect once** (continue without saving), **Trust and connect** (save and continue).

A saved key records the lowercased host, port, key type, fingerprint, the first-seen time and the operating-system user who trusted it.

8.5 Terminal behavior

One xterm.js instance per tab, living outside the interface framework so a tab can move between panes with its screen and scrollback intact. Configuration: WebGL renderer (falling back to the DOM renderer if WebGL fails or the context is lost), Unicode 11 width tables, search, and web-link addons; Option acts as Meta on macOS; right-click does not select words.

- **Fit.** The grid fits the pane. Size changes are debounced 50 ms and, when the column or row count changed, a window-change request goes to the server. With a fixed size the grid is locked and never fits.
- **Links.** Plain URLs and OSC 8 hyperlinks open in the system browser on click; only http(s)-style links are active.
- **Copy and paste.** Copy: Cmd+C on macOS, Ctrl+Shift+C elsewhere, when text is selected. Paste: Cmd+V or Ctrl+Shift+V, or the browser paste event. Copy on select (off by default) copies as soon as a selection exists. Right-click (setting on by default): copies and clears the selection if there is one, otherwise pastes.
- **Multi-line paste.** If the text, ignoring trailing newlines, contains a line break and `confirmMultilinePaste` is on, a confirm dialog titled "Paste multiple lines?" says "You are pasting N lines. Each line may run as a command." followed by the first 600 characters (with an ellipsis if longer); button **Paste**. The terminal wraps accepted text in bracketed-paste markers when the remote program enabled that mode.
- **OSC 52.** A remote program may set the local clipboard only when the connection's `osc52` option is on; the payload is base64 text; a query (`?`) is ignored. When off the sequence is swallowed.
- **Bell.** `visual`: the terminal gets a `bell` style for 150 ms; `audible`: a short 880 Hz tone (80 ms, low gain); `off`.
- **Search.** Ctrl or Cmd+Shift+F toggles a search bar at the top of the pane: input "Find in terminal", a `.*` chip (regular expression), an `Aa` chip (match case), previous and next buttons and a close button. Enter finds next, Shift+Enter previous, Escape closes and returns focus to the terminal. Invalid regular expressions are ignored. Matches are highlighted.
- **Zoom.** Ctrl or Cmd with `=`, `-`, `0` while the terminal has focus changes the tab's zoom by one font point (range −8 to +24; 0 resets). The font size is the profile size plus the zoom, never below 6. Zoom is per tab; the tab menu can save the resulting size on the connection as the option `terminal_font_size`, which later opens start from; resetting zoom and saving removes it.

- **Keys passed to the application, not the shell.** With Ctrl (Cmd on macOS): K, L, W, N, B, D, Tab; and Ctrl+Tab and Ctrl+Shift+Tab everywhere.
- **Reconnect banner.** A second session in the same tab prints a dim line `[reconnected at HH:MM:SS]` (local time) before the new output.

8.6 Terminal appearance

The effective terminal settings are the global defaults (Settings > Terminal) overlaid with the spec of the terminal profile that applies (the connection's own `terminal_profile`, else the nearest folder's). Fields:

Field	Default	Range and notes
fontFamily	`Menlo` (macOS), `Cascadia Mono, Consolas` (Windows), `DejaVu Sans Mono` (other)	Always followed by `monospace` as fallback. The picker offers the installed ones among 24 common monospace fonts
fontSize	13	6 to 48
fontWeight	normal	light (300), normal, medium (500), semibold (600), bold
lineHeight	1.15	0.8 to 2
scheme	OkRemote Dark	Name of a built-in or imported color scheme (Appendix C)
cursorStyle	block	block, underline, bar
cursorBlink	true	
scrollback	10000	0 to 200000 lines
opacity	1	0.5 to 1; scales the background color's alpha
bell	visual	off, visual, audible
copyOnSelect	false	
rightClickPaste	true	
confirmMultilinePaste	true	
highlightRules	empty	Stored but not applied in 0.1.0

The terminal editor (used in Settings > Terminal and the profile dialog) shows the form on the left and a live 64 by 8 preview terminal on the right, fed a fixed sample (a prompt, an `ls -la` listing, red, yellow and cyan log words, and the 16 ANSI colors as squares). **Import...** reads a color scheme file (section 14.5) and selects it.

8.7 Session logging

When `session_log` is on, every byte the server sends is also written, raw, to `<folder>/<safe name>-YYYYMMDD-HHMMSS.log`, where the folder is `session_log_dir` or else `<config>/logs`, and the safe name is the connection name with every character other than letters, digits, `-`, `_` and `.` replaced by `_`. With `session_log_timestamps` each line starts with `[YYYY-MM-DD HH:MM:SS]` in local time. The log path shows in the tab tooltip and the status bar says "logging". Logs can contain secrets typed into the session; the editor warns about it.

8.8 Tab states and overlays

A pane body shows one terminal per tab (only the active one visible). While a tab is `connecting`, a centered "Connecting…" label shows. When a tab is `closed` or `error`, a translucent overlay covers it with a box: a bold "Connection failed" or "Disconnected", the error text in the error color if any, and two buttons: **Reconnect** (primary, autofocused, with a refresh icon) and **Close tab**. Reconnect on a tab that is still connected closes the live session first, prints `[session closed]`, and opens a new one; on a connecting tab it does nothing.

The tab status dot is the state color, or the degraded color when the tab's route transport is not up and not idle.

9. RDP sessions

An RDP connection opens in one of two ways, chosen when the connection is activated: the built-in client in a tab, or a platform client in its own window.

9.1 Choosing the client

1. If the resolved `rdp_client_path` is non-empty, the connection uses that external client (section 9.5) and the tab opened for it closes again at once while the launcher runs instead.
2. Otherwise the tab is created and the built-in client connects. (The tab's controller asks the core to open it; if the core answers "external" the tab is closed and the launcher path is used instead.)

9.2 Built-in client: connect sequence

1. Credentials: if the user name or password is blank after inheritance, a credentials dialog asks for them for this launch only (section 8.2). Cancelling closes the tab with the message "<name> was not opened: no credentials were entered".
2. Routing: if a tunnel profile applies, an ephemeral local forward is started (section 11.2) and the client connects to `127.0.0.1:<port>`.
3. The engine runs on its own operating-system thread with its own single-threaded async runtime (the sign-in step cannot move between threads). The UI waits up to 45 seconds for the first event; a timeout closes the engine and fails with "timed out waiting for the server".
4. TCP connect with a 15-second limit and no-delay set. Then the X.224/RDP negotiation, then a TLS upgrade, then sign-in. With `rdp_nla` true the client requires CredSSP (NTLM; Kerberos is not available and reports so). With `rdp_nla` false TLS-only security is also enabled. Standard RDP security is never enabled.
5. Identity pinning: the SHA-256 of the server's TLS public key (lowercase hex) is compared with the value stored in the workspace host-key table under key type `rdp-tls` for the host name and port as typed by the user (not the local forward address). A mismatch aborts with "the server's identity changed (expected X, got Y); connection blocked in case someone is intercepting it". With no stored value the fingerprint is saved after a successful connect (trust on first use). Quick-connect RDP stores it in `known_hosts.json`.
6. Client parameters: client name `okRemote`, 32-bit bitmaps with lossy compression and the standard codec set, K64 bulk compression, LAN connection type, enhanced 101/102-key keyboard with 12 function keys, no audio, no server-side application mode, server pointer enabled. Initial desktop size is the tab's pixel size (width at least 320, height at least 240 in the core, at least 640 by 480 in the UI).
7. Failure text: any error mentioning logon, CredSSP, access denied or `0xc000006d` becomes "sign-in was rejected; check the user name, password and domain"; others become "connection failed: <detail>".

Options the built-in client honors: only `rdp_nla`. The others (scaling mode, width and height, color depth, scale factor, clipboard, drives, printers, admin session, full screen) apply to platform clients only. The built-in client always resizes the remote desktop to the tab and scales the picture to fit.

9.3 Built-in client: session behavior

- **Picture.** The engine copies each changed rectangle out of its frame buffer as tightly packed RGBA with alpha forced to 255, merging all changes from one batch into their bounding rectangle, and sends one message per batch.
- **Canvas.** The UI keeps one 2D canvas (no alpha) per tab sized to the remote desktop and draws each rectangle with `putImageData`. It scales the canvas with CSS to fit the tab, preserving aspect ratio (largest integer pixel size that fits), centered, black background.
- **Resize.** When the tab size changes, the UI waits 250 ms of quiet, then sends the new size if it differs from the last one sent. The engine rounds it to a size the protocol allows and sends a display-control layout. If the server cannot resize yet the request is dropped. When the server re-activates at a new size, the engine rebuilds its frame buffer and sends a resized event followed by a full repaint.
- **Cursor.** Default cursor, hidden cursor, or a custom bitmap with hotspot (converted to a CSS `url(...) x y, default` cursor).
- **Mouse.** Moves are coalesced to one per animation frame. Coordinates are mapped from canvas pixels to remote pixels and clamped to the desktop. Buttons: 0 left, 1 middle, 2 right. Wheel: vertical and horizontal separately; the delta is `-deltaY` (or `deltaX`) times 1.2, times 40 for line mode or 400 for page mode, clamped to ±255 (one notch is about 120). The context menu is suppressed over the canvas.
- **Keyboard.** Key events are consumed (default action prevented, propagation stopped, so no application shortcut fires while the canvas has focus). Each physical key maps by `KeyboardEvent.code` to a set 1 scan code plus an extended flag; Command on a Mac is sent as Left Control. Keys with no code but a one-character `key` are sent as Unicode events. IME composition events are ignored. The canvas takes focus on click and when the session connects. On blur every held key is released on the remote side.
- **End.** A close message carries a reason string. Reasons starting with closed, server, user, logged or disconnected are normal and show no error; any other reason is shown in the tab's overlay. When the window closes, the engine shuts down gracefully, the session is removed and its ephemeral forward is stopped.

Scan code table (code, extended): Escape 01; digits 02 to 0B; Minus 0C; Equal 0D; Backspace 0E; Tab 0F; letter rows Q to P 10 to 19, A to L 1E to 26, Z to M 2C to 32; BracketLeft 1A; BracketRight 1B; Enter 1C; ControlLeft 1D; Semicolon 27; Quote 28; Backquote 29; ShiftLeft 2A; Backslash 2B; Comma 33; Period 34; Slash 35; ShiftRight 36; NumpadMultiply 37; AltLeft 38; Space 39; CapsLock 3A; F1 to F10 3B to 44; NumLock 45; ScrollLock 46; numpad 7 8 9 - 4 5 6 + 1 2 3 0 . at 47 to 53; IntlBackslash 56; F11 57; F12 58.

Extended: NumpadEnter 1C, ControlRight 1D, NumpadDivide 35, PrintScreen 37, AltRight 38, Home 47, ArrowUp 48, PageUp 49, ArrowLeft 4B, ArrowRight 4D, End 4F, ArrowDown 50, PageDown 51, Insert 52, Delete 53, MetaLeft 5B, MetaRight 5C, ContextMenu 5D.

9.4 Wire format from the core to the interface

One binary message per event: a tag byte, then little-endian fields.

Tag	Meaning	Fields
1	Frame	x, y, width, height (u16 each), then width×height×4 RGBA bytes
2	Resized	width, height (u16)
3	Cursor default	none
4	Cursor hidden	none
5	Cursor bitmap	width, height, hotspot x, hotspot y (u16), then RGBA
7	Closed	UTF-8 reason

Input goes the other way as JSON objects with a `t` field: `move {x,y}`, `button {x,y,button,down}`, `wheel {x,y,delta,horizontal}`, `key {code,extended,down}`, `unicode {ch,down}`, `resize {width,height}`, `close`.

9.5 Platform clients (`rdp_client_path` set, or the legacy path)

The launcher writes a `.rdp` file (never containing the password) to a private temp file and starts the client. The file, one `key:type:value` per line with CRLF line endings:

Line	Value
full address:s	`host:port`
username:s	`DOMAIN\user` when a domain is set and the user has no `\` or `@`, else the user
screen mode id:i	2 when `rdp_fullscreen`, else 1
desktopwidth:i, desktopheight:i	Only for scaling fixed or scroll with both sizes set

Line	Value
dynamic resolution:i	1 when scaling is dynamic
smart sizing:i	1 when scaling is smart
desktopscalefactor:i	Only when not 100; clamped 100 to 500
session bpp:i	`rdp_color_depth`
redirectclipboard:i, redirectprinters:i	0 or 1
drivestoredirect:s	`*` when drives are on, else empty
enablecredsspsupport:i	`rdp_nla`
administrative session:i	`rdp_admin`
prompt for credentials:i	0
authentication level:i	2
autoreconnection enabled:i	1

- **Windows:** `mstsc.exe <file>` (or the custom client). If a password and user exist, a Credential Manager entry of type domain password named `TERMSRV/<host>`, persisted for the logon session, is written first unless one already exists (an existing one is left untouched); it is deleted 30 seconds later if okRemote wrote it. If it could not be written, the password is copied to the clipboard.
- **Linux and other Unix:** FreeRDP, found as `xfreerdp3`, `sdl-freerdp3`, `xfreerdp` or `wlfreerdp` (or the custom path), error "FreeRDP was not found; install the freerdp3-x11 or freerdp2-x11 package". Version 3 is detected by a `3` in the program name, an `sdl-` prefix or `--version`.
- **macOS:** FreeRDP is used only when the custom path names a FreeRDP binary, or when neither Windows App nor Microsoft Remote Desktop is installed (searching `sdl3-freerdp`, `sdl-freerdp3`, `sdl-freerdp`, `xfreerdp3`, `xfreerdp` on PATH and in Homebrew and MacPorts folders). Otherwise the `.rdp` file is opened with `open -a "Windows App"` (or the custom app, or Microsoft Remote Desktop); the password is copied to the clipboard. With no handler: "no application is installed to open .rdp files; install Windows App from the App Store".

FreeRDP arguments, in order: `/v:host:port`; `/u:user`; `/d:domain`; `/from-stdin:force` when both a password and a user exist (the password is then written to its standard input plus a newline); `/cert:tofu` (v3) or `/cert-tofu` (v2); `/dynamic-resolution`, `/smart-sizing` or `/size:WxH`; `/f` for full screen; `/scale-desktop:N` when not 100; `/bpp:N`; `+clipboard` or `-clipboard`; `/drive:home,~`; `/printer`; `-sec-nla` when NLA is off; `/admin`; `/title:<name>`. Output goes to a log file (`~/Library/Logs/OkRemote/freerdp-last.log` on macOS, `~/.local/state/okremote/freerdp-last.log` elsewhere) after one line stating whether okRemote itself could reach the host. The launcher waits up to 5 seconds; if the client exits with an error, the failure shows the last three error lines plus a hint (local-network permission on macOS, logon failure, or connection failure) and the log path.

10. VNC, web and external-tool sessions

These three connection types do not run in a tab. They launch a program or open a window, and the main window shows a toast with the result (client name, route, and any clipboard notice).

10.1 Rules shared by every launch

1. Resolve options and credentials (section 3, folder inheritance applied).
2. For VNC, ask for a missing password in a dialog (section 8.2) for this launch only; nothing is saved.
3. If a tunnel profile applies, start an ephemeral local forward and use `127.0.0.1:<port>` as the address (section 11.2).
4. A password is never placed on a command line. Where the client cannot receive it safely, the app copies the password to the clipboard and clears it after `clipboard_clear_seconds` (default 30, setting) if the clipboard still holds it. The toast says which client could not receive it.
5. If the client is a tracked child process, its forward stops when the process exits. Temporary files (private mode 0600 on Unix, named `okremote-<uuid>.<ext>` in the OS temp folder) are deleted 30 seconds after launch.

10.2 VNC

- **macOS, no custom viewer path:** opens `vnc://[user@]host:port` with the system `open` command, so Screen Sharing handles it. The password cannot be passed; it is copied to the clipboard. Failure message: "Screen Sharing could not be opened".

- **Otherwise:** the viewer is `vnc_client_path` if set, else the first found of `vncviewer`, `tigervnc`, `xtigervncviewer` on PATH, then fixed install paths (TigerVNC, RealVNC and UltraVNC on Windows, the TigerVNC app on macOS). Not found: "no VNC viewer was found; install TigerVNC or set the viewer path on the connection".
- Arguments: `host::port` first; `-PasswordFile <temp file>` when a password exists and the viewer is TigerVNC-like (any viewer whose path does not contain `realvnc`); `-ViewOnly` when `vnc_view_only`; `-RemoteResize=1` when `vnc_remote_resize` and the viewer is TigerVNC-like. For a RealVNC viewer the password is copied to the clipboard instead.
- The password file holds the 8-byte VNC obfuscation of the password: the first 8 bytes of the password (zero padded) encrypted with single DES under the fixed key `E8 4A D6 60 C4 72 1A E0` (standard DES bit order, as `vncpasswd` writes). Test vector: password `password` gives `DB D8 3C FD 72 7A 14 58`.

10.3 External tool

The connection's `ext_command` is a command line.

1. Split into arguments honoring single and double quotes; the quotes are removed; an empty quoted string is an argument; an unbalanced quote is an error ("unbalanced quotes in command"). A missing or blank command is an error ("this connection has no command line").
2. Substitute in every argument, case-insensitively: `%NAME%` (connection name), `%HOST%`, `%PORT%` (after tunnel rewriting), `%USER%` and `%DOMAIN%` (empty when unset). `%PASSWORD%` anywhere is refused with a message telling the author to read `$OKREMOTE_PASSWORD` or stdin.
3. The program (first argument, substituted) is started directly, not through a shell. Working directory is `ext_working_dir` when set.
4. Environment: `OKREMOTE_HOST`, `OKREMOTE_PORT`, `OKREMOTE_USER` (when known). Password by `ext_password_via`: `env` (default) sets `OKREMOTE_PASSWORD`; `stdin` writes the password plus a newline to the tool's standard input; `none` passes nothing. With any mode other than stdin, standard input is closed.
5. The process is tracked, so an ephemeral forward ends with it.

10.4 Web (HTTP and HTTPS)

URL: if the stored host already contains `://` it is the URL. Otherwise `scheme://host[:port][/path]`, where the port appears only when it differs from the protocol default and `http_path` gets a leading `/` if missing.

1. `http_open_in = system` and no tunnel: open in the system browser; toast client "system browser".
2. HTTPS with no tunnel: ask `curl --head` (6 second limit, 4 second connect limit; uses the system trust store) whether the certificate is accepted. Exit codes 60, 51, 58, 59, 77, 83, 90 or 91 mean untrusted; 35, 54, 56, 1 or 4 mean the built-in viewer may not manage the secure connection. In either case open the system browser instead and toast the reason plus "opened in your browser instead, where you can review the warning". If curl is missing or the check does not fail, continue.
3. Otherwise open a new okRemote window: label `web-<token>`, inner size 1280 by 860, title `<connection name> — <url>`, with the zoom set to `http_zoom` clamped to 0.25 through 5 when it is not 1.0. With a tunnel it uses the SOCKS5 proxy from section 11.2.
4. If both a user name and a password are set, the password is copied to the clipboard (cleared after the usual delay) and the toast says it was copied for the sign-in page. There is no automatic form filling and no HTTP basic-auth injection.
5. When the window is destroyed its ephemeral proxy forward stops.

Web windows load remote pages and receive no access to the application's command interface (section 5.1).

11. Tunnels and jump chains

11.1 Concepts

- A **tunnel profile** (section 3.6) is an ordered chain of SSH connections from the same workspace plus optional forwards. Each hop authenticates with its own credentials and its own host-key check.
- A **shared transport** is one live SSH connection through a chain. Its key is `<workspace id>:<hop ids joined by commas>`, so every user of the same chain shares one transport (a session, a forward, a web proxy). The label shown in the interface is the hop names joined by `→`.
- A **forward** listens somewhere and moves bytes over a transport. Each running forward holds a lease on its transport.

- Every forward has an **owner** string used to stop groups of forwards together: `profile:<workspace>:<profile id>` for a profile started by hand or by auto-start, `rdp:<id>`, `launch:<id>` or `web:<id>` for ephemeral forwards belonging to one session.

11.2 How a connection uses a profile

A connection uses a profile when its resolved `tunnel_profile` option is a profile id. The value `none` (or empty) on a connection or folder switches inheritance off. A profile with an empty chain is an error ("the tunnel profile has no SSH hops"); a hop that is not an SSH connection is an error naming it.

Connection type	Route
SSH	Get the shared transport (connecting the whole chain if needed). If the connection is itself the last hop of the chain, use that transport directly. Otherwise open a new SSH connection from the last hop through a direct-tcpip channel, then the shell on that
Embedded RDP, platform RDP, VNC, external tool	Start an ephemeral local forward `127.0.0.1:0 → host:port` (auto flag set, label `<name> (<PROTO> host:port)`), wait for the transport to connect so prompts appear first, then point the client at `127.0.0.1:<assigned port>`
Web in an okRemote window	Start an ephemeral dynamic (SOCKS5) forward on `127.0.0.1:0` and give the web view `socks5://127.0.0.1:<port>` as its proxy, so TLS and SNI use the real host name
Web in the system browser	The profile is ignored for routing: when a profile applies, the connection opens in the okRemote window instead

Ephemeral forwards stop when their client exits (tracked child process), when the web window is destroyed, or when the embedded RDP session ends. If the OS owns the client and cannot be tracked (macOS Windows App, Screen Sharing), the forward stays until stopped in the Tunnels panel.

11.3 Auto-start

When a connection with a profile opens and the profile has `auto_start` and at least one forward, the forwards start once (skipped if any forward owned by `profile:<ws>:<id>` is already running). Forwards that bind beyond loopback are skipped here and need an explicit start with confirmation.

11.4 Forward kinds

- **Local.** Bind `bind_addr:bind_port` (empty address means `127.0.0.1`; port 0 picks a free one). Each accepted socket gets its own task that opens a direct-tcpip channel from the last hop to `remote_host:remote_port` and copies both ways, with TCP no-delay on.
- **Dynamic.** Same listener, but each client speaks SOCKS5: no authentication (a client that does not offer method 0 gets `05 FF`), CONNECT only (other commands get reply 7), address types IPv4, domain name and IPv6 (others get reply 8). Replies: 0 success, 1 general failure when the transport is down, 5 refused when the channel cannot open. Reply bytes are `05 <code> 00 01 00 00 00 00 00 00`.
- **Remote.** Ask the server to listen on `remote_bind_addr:remote_bind_port` (empty means `127.0.0.1`; port 0 lets the server assign one and the assigned port is shown). Incoming connections open a local socket to `local_host:local_port`. If the server refuses, the error is "server refused remote forward". The listener is re-requested after every reconnect and cancelled when the forward stops.
- A forward that exposes beyond loopback is refused by the core unless the caller passes the allow flag, which the interface sets only after the user confirms.

Each running forward reports: id, label, kind, listen address, target, transport key and label, transport health, count of active connections, auto flag and owner.

11.5 Transport health

State	Meaning
idle	Not connected, no users
connecting	First connection or a replacement is being made
up	Connected
reconnecting (attempt, error)	Lost; trying again

State	Meaning
failed (error)	Connection failed, or 10 attempts used up

A monitor task wakes every 3 seconds per transport. With users and a dropped connection it reconnects: attempt n waits `min(2^min(n,6), 60)` seconds before the next try (2, 4, 8, 16, 32, then 60), up to 10 attempts, stopping early if all users leave. With no users for more than 60 seconds it closes the connection and returns to idle. Each hop connect has a 20-second timeout; keepalive packets go out every `keepalive_seconds` (0 turns them off) and the link is dropped after 3 missed. Every health or lease change emits the `tunnels-changed` event so the interface refreshes.

On application exit all forwards stop and all transports close.

11.6 Tunnels dialog

Opened from the top-bar menu (Tunnels…) or the status bar's forwards button; a workspace's context menu opens it for that workspace. A wide modal titled **Tunnels** with a single **Close** button.

1. A row with a **Workspace** select (unlocked workspaces) on the left and **New profile** on the right.
2. **Profiles** table: name with the hop names joined by arrows under it ("no hops" when empty); the forwards as monospace lines (`-L addr:port → host:port` , `-D addr:port (SOCKS5)` , `-R addr:port → host:port`), each with a warning badge `network` when exposed ("jump chain only" when none); and buttons **Start/Stop** and **Edit**. Start is disabled when there are no forwards. If any forward exposes beyond loopback a confirm dialog asks "Expose a forward to the network?" with the confirm label **Start anyway** (danger style) before starting. Success shows a toast listing the listen addresses.
3. **Running** table: health dot, label (plus an `auto` badge), `kind listen → target` , `<transport label> · N active` with `· retry N` or `· <error>` as relevant, and **Stop**.
4. **SSH transports** table: health dot, label, `<state> · N users` .

When no profiles exist the hint reads: a tunnel profile is a chain of SSH jump hosts plus optional forwards, attached to a folder or connection under Edit → Tunnel.

11.7 Profile editor

A wide modal titled `Edit <name>` or `New tunnel profile`. Footer: **Delete** (only for saved profiles, danger style, confirm "Delete tunnel profile?"), spacer, **Cancel**, **Save**.

- **Name** (autofocus).
- **Jump chain**: an ordered list; each item shows `name (host)` or "missing connection", with up, down and remove icon buttons. An **Add a hop…** select lists every SSH connection in the workspace as `Folder path / name (host)`.
- **Forwards**: one row per forward with a kind select (Local (-L), Dynamic SOCKS (-D), Remote (-R)), address and port inputs, a right arrow and target inputs where relevant, and a remove button. Defaults for a new row: local `127.0.0.1:8080 → localhost:80`; dynamic `127.0.0.1:1080`; remote `127.0.0.1:9000 → localhost:9000`. A row that exposes shows a warning "Reachable from the network". **Add forward** appends a local row.
- A checkbox: "Start the forwards whenever a connection using this profile opens".
- Save validates: a name is required; at least one hop is required (toast otherwise).

12. Tabs, panes and the welcome view

12.1 Pane tree

The session area holds a binary tree. A **leaf** pane has an ordered list of tab ids and one active tab. A **split** has a direction (`row` places the two children side by side, `column` stacks them), a ratio (0.5 at creation) and two children. The tree starts as one empty leaf. One leaf is the **active pane**; clicking anywhere in a pane makes it active (a pane with the active mark shows a highlighted tab bar).

Tab state, per tab: id, title, protocol, workspace id and connection id (or a quick target), session id, state (`connecting` , `connected` , `closed` , `error`), error text, folder color, route label, zoom, terminal profile id, OSC 52 flag, fixed size, log file path and the current column and row count.

12.2 Tab bar

Each pane has its own tab bar: a horizontal strip, minimum height 34 px, background `--bg-3` , bottom border, scrolling sideways when crowded. A tab is 110 to 230 px wide with a 3 px top border in the folder color (transparent when none) and these parts left to right: a status dot (section 5.8), a protocol icon (13 px), the title (ellipsized), a small tunnel icon

when the session is routed (tooltip `Through <route>`), and a close button (an x icon, label `Close <title>`). The active tab is raised. Tooltip: `<title> — via <route>` and, when logging, a second line `Logging to <path>`.

At the right end of the bar, when the pane has tabs, two icon buttons: **Split right** (tooltip `Split right (Ctrl+D)`) and **Split down** (`Split down (Ctrl+Shift+D)`). Both first make this pane active.

Mouse: click activates; middle-click closes; right-click opens the tab menu (12.5); dragging a tab (type `application/x-okremote-tab`) over another tab shows a drop mark before it and dropping moves the tab to that index, possibly in another pane; dropping on empty bar space moves it to the end.

12.3 Splitting

Split right or down makes the active pane a split whose first child keeps the pane's tabs and whose second child is a new empty pane, which becomes active. If the original pane had more than one tab, its active tab moves into the new pane. The divider between the two children is 4 px wide and draggable; the ratio is clamped between 0.15 and 0.85. When a pane becomes empty it is removed and its split collapses into the surviving sibling, except that a lone root pane stays even if empty.

12.4 Opening, closing, cycling

- Opening a connection adds a tab at the end of the active pane (or the pane that asked, such as a welcome view button) and activates it. Opening the same connection twice opens two tabs.
- Closing: if the tab is `connected` and the setting `confirm_tab_close` is on (default), a confirm dialog asks "Close this session?" / "<title> is still connected." with button **Close** and a checkbox "Don't ask again" that turns the setting off. Tabs in other states close at once. Closing frees the terminal or RDP canvas, ends the session and removes the tab; the neighboring tab becomes active.
- Ctrl+Tab and Ctrl+Shift+Tab cycle within the active pane, wrapping around. Ctrl or Cmd+W closes the active tab.
- The list of open tabs (workspace id and connection id, or the quick target) is saved to the `last_tabs` setting whenever a tab opens or closes.

12.5 Tab menu

Item	Behavior
Reconnect	Disabled while connecting. See 8.8 and 9.3
Duplicate	Opens the same saved connection in a new tab; disabled for quick targets
Zoom in, Zoom out, Reset zoom	Terminal font size zoom (8.5)
Save font size for this connection	Stores the zoomed size on the connection; disabled for quick targets
Edit connection	Opens the editor drawer; disabled for quick targets
Close	Same as the close button

Separators sit after Duplicate, after the zoom group and after Edit connection.

12.6 Welcome view

An empty pane shows a centered welcome panel: the heading `okRemote`, the line "Pick a connection in the tree, or press Ctrl+L to connect to a host by address.", then:

- A **restore bar** when no tab is open, the `restore_tabs` setting is `ask` and `last_tabs` is not empty: "N tab(s) was/were open last time." with buttons **Restore tabs** and **Dismiss** (which clears the saved list).
- Two columns: **Favorites** (every pinned connection; empty text "Pin a connection from its context menu to see it here.") and **Recent** (the 10 most recent connections that still exist, each showing `<PROTOCOL> · <host>`; empty text "Connections you open appear here."). Each entry is a button that opens the connection in this pane.
- Three buttons: **New connection**, **Import**, **Command palette** (with a `Ctrl+K` key hint).

Recents are a list of up to 25 workspace and connection id pairs with timestamps, newest first, one entry per connection, saved in settings; only saved connections are recorded, not quick targets.

12.7 Restoring and auto-opening

- `restore_tabs = always`: after all startup workspaces are open, the saved tabs reopen one after another (quick targets too). `ask`: the welcome bar above. `never`: nothing.

- When a workspace unlocks, every connection in it with `auto_open` true opens, unless a tab for that connection already exists.
- `startup_mode`: `last` opens every workspace that was open last time, `pinned` opens only the pinned workspace, `chooser` opens none (the first-run or empty view shows). Files named on the command line open first in every mode, and a pinned workspace is always opened first unless the mode is `chooser`.

13. Quick connect, command palette, favorites and recents

13.1 Quick connect

The top-bar field (section 5.3) accepts the forms in section 3.10. **Connect** opens a *quick target*: a connection that exists only for this session. Its tab title is the full URL for web targets, otherwise `user@host:port` with absent parts omitted. It has no workspace, so it uses the device host-key store (`known_hosts.json`), no folder credentials and no tunnel profile; a user name from the text is used, and a missing user name or password follows the usual prompts (section 8.2). Web quick targets open in an okRemote window. Quick targets are remembered in `last_tabs` but never in recents.

Save turns the text into a stored connection in the insertion point (section 6.9) and opens the editor so the user can finish it.

Ctrl or Cmd+L focuses the field (the sidebar need not be open).

13.2 Command palette

Opened by Ctrl or Cmd+K or the top-bar button. A modal sheet near the top of the window, `min(640px, 94vw)` wide, rounded 10 px, containing a text input (placeholder "Type a connection or command…", exposed as a combobox) and a list of options below it. Each option shows an icon, a label, an optional dimmer detail line and an optional key hint.

Contents. Every connection of every unlocked workspace (label = name; detail = `<PROTOCOL> · <host>` plus `· <folder path>`; icon = protocol), and these commands, with the hint from the user's key bindings where one exists:

1. New connection (Mod+N)
2. New folder (Mod+Shift+N)
3. Quick connect (Mod+L)
4. Filter the tree (Mod+F)

5. Lock workspaces (Mod+Shift+L)
6. Open or create a vault
7. Import connections
8. Tunnels
9. Terminal profiles
10. Host keys
11. Audit log
12. Settings
13. Split pane right (Mod+D)
14. Split pane down (Mod+Shift+D)
15. Close tab (Mod+W)
16. Toggle sidebar (Mod+B)
17. Restore tabs from last session
18. Switch light / dark theme

Matching. Score of a text against the lowercase query: empty query 1; starts with the query 3; contains it 2; query letters appear in order (subsequence) 1; otherwise 0 (excluded). A connection's score is the larger of twice its name score and the score of `host path tags`. Results sorted by score, highest first.

Order. With a query: up to 40 matching connections, then matching commands. Without a query: all commands in the order above, then the first 30 connections in workspace and tree order. An empty result shows "Nothing matches.".

Keys. Up and down move the highlight (it starts on the first row and resets to it when the query changes), Enter runs the highlighted option, Escape or a click outside closes. Hovering an option highlights it; clicking runs it. The palette closes before the action runs. The theme command flips between `light` and `dark` based on the theme currently showing.

13.3 Favorites

A connection is a favorite when its own option `pinned` is true. Set by the editor checkbox or the menu item *Pin to favorites*. Shown in the sidebar strip (6.3) and the welcome view (12.6). Pinning never inherits from folders.

13.4 Recents

Section 12.6. Opening a saved connection from anywhere (tree, palette, favorites, welcome, tab menu Duplicate, restore) records it; recents are shown only for connections that still exist.

14. Import and export

14.1 Import dialog

Opened from the top-bar menu, the palette, a workspace's context menu or the welcome view. A wide modal titled **Import connections**. If no unlocked, writable workspace exists it shows the error text "Open and unlock a workspace first; imports go into a workspace." Footer: **Cancel** and a primary button that reads **Preview**, then **Import N connections**.

Step 1, choose the source (a select, default the first entry), read the hint under it, then pick a file:

Source	File	Hint shown
XML connection tree (.xml)	`.xml`	Folders, connections, credentials and inheritance. You are asked for the file password if one is set
OpenSSH config	optional (default `~/.ssh/config`, shown as the placeholder)	Hosts, users, keys, ProxyJump chains and forwards. Leave the file empty to use ~/.ssh/config
PuTTY saved sessions	none	Read from this computer (Windows registry or ~/.putty/sessions)
Remote Desktop Connection Manager (.rdg)	`.rdg`	Groups and servers. Stored passwords are DPAPI-protected and cannot be imported
CSV	`.csv`, `.txt`	Columns: name, host, port, protocol, username, password, domain, folder, tags, notes

Source	File	Hint shown
okRemote JSON export	`.json`	A plain export from okRemote (without secrets)

Preview is disabled until a file is chosen (except for the two sources that need none) and, when a password is being asked, until it is filled. If the file needs a password the core answers `password_required`; the dialog then shows a **File password** field ("Used only to decrypt; passwords go straight into the vault's encrypted fields.") with a toast "This file has a password. Enter it to continue."; a wrong one toasts "That password is not correct.". Other errors toast for 8 seconds.

Step 2 shows the preview: four counters (connections, folders, tunnel profiles, with credentials); a heading `N items need attention` with the warnings as a list (`<item>: <message>`); a collapsed `N notes` section with the info messages; a **Into workspace** select; an **Under folder** select (Top level, then every folder by path); and **Wrap in a new folder** (default `Imported from <source>`, empty means import directly). The primary button imports and then toasts `Imported N connections, M folders` plus `, K tunnel profiles` for 7 seconds, refreshes the tree and closes.

14.2 Preview and commit

Preview parses the file entirely in memory and returns counts and the report; the parsed records wait under a random token. Nothing is written until Commit. Commit fails with "the import preview expired; run it again" if the token is gone. Commit order: create the wrap folder (if any) under the chosen parent; give every top-level imported folder and connection that parent; save folders (parents always come before children), each with its credentials; save connections with credentials; save tunnel profiles; record one audit entry `import` with the source name and the three counts. Imported records are saved as new (version 0). A tunnel profile that references imported connections uses the ids those connections were saved with.

Input limits: 64 MiB per file ("file is too large to import"). Text files may be UTF-8 with or without a byte-order mark, or UTF-16 little-endian with a mark.

A report item has an item name, a level (`warning` or `info`) and a message.

14.3 Source mappings

CSV. The first row is the header, matched case-insensitively with these synonyms: name (`name`, `display name`, `displayname`); host (`host`, `hostname`, `address`, `server`; required, else "CSV needs a 'host' column"); port; protocol (`protocol`, `type`); user (`username`, `user`); `password`; `domain`; folder (`folder`, `path`, `group`); tags (`tags`, `tag`, split on `;` or `,`); notes (`notes`, `description`, `descr`). Cells are trimmed and rows may be ragged. A row with no host is skipped with a warning naming its line. Unknown protocols become SSH with a warning. The name defaults to the host. A port equal to the default is dropped. The folder column is a path split on `/` or `\`; folders are created as needed and shared across rows. If any password was present a warning says the file holds plaintext passwords and should be deleted.

OpenSSH config. One connection per concrete `Host` alias (an alias without `*`, `?` or `!`). Keywords are case-insensitive, `=` or whitespace separates keyword and value, quotes are stripped. The first value given for a keyword wins; settings from `Host *` blocks fill what a host leaves unset. `HostName` (with `%h` replaced by the alias) is the host, `Port` the port (22 dropped), `User` the user name, `IdentityFile` the key file (and sets authentication to automatic), `ServerAliveInterval` the keepalive. `Match` blocks and their contents are skipped with a warning; `Include` lines are not followed (warning); `ProxyCommand` warns. `Host *` `User` and `IdentityFile` are applied to every imported host (one info note). `ProxyJump` (comma-separated hops, `none` ignored): each hop that matches an alias uses that connection, any other becomes a new connection tagged `jump` (parsing `user@host:port`), and one tunnel profile named `Jump via <spec>` per distinct chain is created and set as the host's `tunnel_profile`. `LocalForward`, `RemoteForward` and `DynamicForward` become one tunnel profile per host named `<host> forwards` with the host itself as the single hop and `auto_start` on (info note: start it from the Tunnels panel); a bind address of `*` becomes `0.0.0.0`, an empty one `127.0.0.1`; unparsable forwards warn.

PuTTY. Windows: every key under `HKCU\Software\SimonTatham\PuTTY\Sessions`; others: files in `~/.putty/sessions` (`key=value` lines). Names are percent-decoded. Only `Protocol=ssh` sessions import (others warn and are skipped); the session named `Default Settings` and sessions with no `HostName` are skipped (info). `user@host` in the host field splits off the user; `UserName` wins. `PortNumber` (22 dropped), `PublicKeyFile` as the key file (a `.ppk` extension warns: convert with PuTTYgen), `PingIntervalSecs` as keepalive. A proxy configuration warns that it was not imported.

RDCMan (.rdg). The root element must be `RDCMan`. Each `group` becomes a folder (name from `properties/name`), each `server` an RDP connection (host from `properties/name` or `name`, title from `displayName`, notes from `properties/comment`, port from `connectionSettings/port` with 3389 dropped). `logonCredentials` (unless `inherit` is set to something other than `None`) gives user name and domain. Stored passwords are never imported; one warning counts them.

XML connection tree. A root element `Connections` holding nested `Node` elements; a node with `Type="Container"` is a folder, any other node is a connection. `Name`, `Hostname`, `Port` (default dropped), `Descr` (notes), `Panel` (becomes a tag unless it is `General`), `Username`, `Domain`, `Password`. Protocol: `RDP`, `VNC`, `SSH2`, `HTTP`, `HTTPS`; `SSH1` imports as SSH with a warning; anything else becomes an External entry with a warning and a note. A field marked inherited by `Inherit<Field>="true"` is left unset so the folder supplies it. Display attributes become options: `Colors` (`Colors256`, `Colors15Bit`, `Colors16Bit`, `Colors24Bit`, other = 32) to `rdp_color_depth`; `Resolution` (`FitToWindow` or `SmartSize` to smart scaling, `Fullscreen` to full screen, `ResWxH` to fixed with width and height); `AutomaticResize` to dynamic scaling; `RedirectDiskDrives`, `RedirectPrinters`, `RedirectClipboard`, `UseCredSsp`, `ConnectToConsole`, `VNCViewOnly` to the matching switches. A remote-desktop gateway host is saved into the notes (`RD Gateway: <host>`, warning that gateways are unsupported) unless its usage is `Never`; a non-default PuTTY session name produces an info note.

Encryption of the XML tree file: the root carries `BlockCipherMode`, `KdfIterations` (default 1000), `FullFileEncryption`, `Protected` and `ConfVersion`. Values are base64. GCM layout: salt (16 bytes) followed by nonce (16) followed by ciphertext and tag (16), key from PBKDF2-HMAC-SHA1 over the file password and the salt, AES-256-GCM with the salt as associated data. Older layout (no mode named and version below 2.6): IV (16) then AES-128-CBC ciphertext with PKCS7 padding, key = MD5 of the password. With no file password the built-in password `mR3m` is used. `Protected` decrypts to `ThisIsNotProtected` (built-in password) or `ThisIsProtected` (custom password); another result means a password is required (none given) or wrong (one given). With `FullFileEncryption="true"` the whole body of the root element is one encrypted value that decrypts to the XML of the child nodes. A value that fails to decrypt is skipped with a warning.

14.4 okRemote JSON

The export document is `{ "format": "okremote-export", "version": 1, "exported_at": <RFC 3339>, "workspace": <name>, "secrets_included": false, "folders": [...], "connections": [...], "tunnels": [...], "terminal_profiles": [...] }`, pretty-printed, written atomically. **There is no menu item or button for exporting in 0.1.0**; the command exists for a later interface. Import reads folders, connections and tunnels (not terminal profiles): folders are ordered parents first, folders whose parent is missing move to the top level, connections whose folder is missing move to the top level, versions are reset to 0 and `has_secret` to false, and no credentials are created.

14.5 Color schemes

The terminal editor imports a color scheme file and adds it to the settings list `custom_schemes` (a scheme with the same name is replaced). Format is picked by extension (`.itermcolors`, `.json`, `.yaml` / `.yml`), or by content: leading `<` is iTerm2, leading `{` Windows Terminal, containing `base00` base16, anything else Xresources. All produce xterm.js theme keys (`foreground`, `background`, `cursor`, `selectionBackground`, the 16 ANSI names). A scheme without both a foreground and a background is rejected ("the scheme has no foreground or background color"). The scheme name is taken from the file, else from the file name. Color forms accepted: `#rgb`, `#rrggbb`, `rgb:rr/gg/bb`. iTerm2 maps `Ansi N Color` 0 to 15 and Background, Foreground, Cursor, Cursor Text, Selection colors from floating-point components. Windows Terminal reads the first entry of `schemes` (or the object itself) and maps `purple` to magenta. Base16 maps base00 to background, base05 to foreground, base02 to selection, and ANSI colors per the usual base16 terminal convention. Xresources reads `*foreground`, `*background`, `*cursorColor`, `*colorN` with `#define` aliases. A scheme is considered dark when its background's luminance (`0.2126R + 0.7152G + 0.0722B`) is below 0.5.

15. Dialogs, menus and notifications

15.1 Dialog frame

Every dialog uses one frame: a dimmed backdrop, a centered box (10 px radius, shadow, at most 90% of the window height), a header with the title and a close button, a scrolling body and an optional footer of buttons. Widths: standard `min(460px, 94vw)`, wide

`min(720px, 94vw)`, extra wide `min(1000px, 96vw)`. Clicking the backdrop or pressing Escape closes it (as Cancel). On open, focus goes to the first input, select, text area or primary button; on close, focus returns to what had it. Only one dialog shows at a time; prompts from the connection core (section 8.2) queue separately and appear above it.

15.2 Dialog index

Dialog	Opened by	Width	Section
Unlock	Opening a locked vault, Unlock buttons, double-clicking a locked workspace	standard	15.3
Vault files (Open, Create new)	Menu, palette, sidebar Vault button, first-run screen	wide	15.4
Confirm	Many actions	standard	15.5
Input	New folder	standard	15.5
Bulk edit	Context menu on a multi-selection	standard	15.6
Conflict	A save against a stale record	wide	7.6
Import connections	Menu, palette, workspace menu, welcome view	wide	14.1
Settings	Menu, palette	extra wide	16
Tunnels, tunnel profile editor	Menu, palette, status bar, editor, workspace menu	extra wide, wide	11.6, 11.7
Terminal profiles	Menu, palette, editor, workspace menu, Settings	extra wide	15.7
Trusted host keys	Menu, palette, host-key-changed prompt	extra wide	15.8
Audit log	Menu, palette, workspace menu	extra wide	15.9
About okRemote	Menu	standard	15.10

Dialog	Opened by	Width	Section
Host key prompts, credentials, passwords, keyboard-interactive	Connection attempts	standard or wide	8.2, 8.4

15.3 Unlock

Title `Unlock <workspace name>`. Shows the vault file path in small text, a **Passphrase** field (hidden, autofocus) and a checkbox "Remember on this device (OS keychain)". Buttons **Cancel** and **Unlock** (primary; disabled while empty or busy; reads "Unlocking…" meanwhile). Enter submits. A wrong passphrase shows "That passphrase is not correct." inside the dialog; other errors show their message. On success the dialog closes, the tree loads and auto-open connections start.

15.4 Vault files

Title "Vault files". A two-button segmented control **Open** and **Create new**.

- **Open:** a primary button "Choose a vault file…" (system file picker filtered to `.okvault`) that opens the chosen file and closes the dialog; under a **Recent** heading, one button per remembered path (vault icon, monospace path, an `open` badge and disabled when already open). Opening a file whose vault is locked immediately shows the Unlock dialog; a failure toasts `Could not open <path>: <message>` for 8 seconds.
- **Create new:** fields **Workspace name** (default `Personal`), **File** (monospace path, pre-filled with the default vault path, with **Browse…** opening a save dialog that appends `.okvault` when missing), **Passphrase** (hint "There is no recovery if you forget it. Use a long phrase."), **Repeat passphrase**, and the remember checkbox. Footer **Cancel** and **Create vault** (primary, "Creating…" meanwhile, disabled while path or passphrase is empty). Checks: at least 8 characters ("Use a passphrase of at least 8 characters"), both entries equal ("The passphrases do not match"). The new vault opens unlocked with one empty folder named `Example`; toast `Created <name>`.

15.5 Confirm and input

Confirm: a title, a body of preformatted text (line breaks kept), an optional checkbox, **Cancel** and an OK button whose label and style vary (danger for deletions). The OK button is autofocused. Closing by any other route counts as Cancel. **Input:** a title, one labeled text field with the initial text selected, **Cancel** and **OK**; Enter confirms.

15.6 Bulk edit

Title `Edit N connections`. A **Change** select with five entries: *Tunnel profile*, *Terminal profile*, *Credentials: clear, and inherit from the folder*, *Add a tag*, *Remove a tag*. Under it, the matching control: a select (tunnel: "Inherit from folder", "None", then each profile; terminal: "Inherit", then each profile), a **Tag** text field, or for credentials a warning "The stored credentials on these connections are deleted.". **Apply** sets or removes the option on every selected connection (an empty choice removes the key), adds or removes the tag, or deletes the connections' own credentials, then toasts `Updated N connections` and closes.

15.7 Terminal profiles

Title "Terminal profiles". A **Workspace** select and **New profile** at the top. Below, two columns: a list of the workspace's profiles (an empty list says "No profiles yet. Connections use the app default from Settings → Terminal.") and, for the selected profile, a **Profile name** field, the terminal editor (8.6) and buttons **Delete** (danger; confirm "Delete profile?" "<name> will be removed; connections using it fall back to the app default.") and **Save profile** (toast "Profile saved" for 2 seconds). A new profile starts as a copy of the global terminal settings and gets a new UUID. Saving re-applies terminal settings to every open tab.

15.8 Trusted host keys

Title "Trusted host keys". A **Store** select (one entry per unlocked workspace, labeled `<name> (shared with the workspace)`, plus "This device (quick connect)") and a filter box. A table with columns Host (`host:port`), Type, Fingerprint (monospace), First seen (date and who trusted it) and a **Remove** button (danger, small) per row. RDP identity pins appear with type `rdp-tls`. Empty text: "No trusted keys yet. Keys are recorded the first time you trust a server."

15.9 Audit log

Title "Audit log". A row of controls: workspace select, an action select ("All actions" plus every distinct action found), a filter box (matches actor, action, item name and the detail JSON). A note "Secret fields are recorded as changed, never as values.". A scrolling table: **When** (UTC `YYYY-MM-DD HH:MM:SS`, monospace), **Who**, **Action**, **Item** (the name of the connection, folder or tunnel profile, else the first 8 characters of its id). The core returns

the newest 5,000 entries; the table shows at most 1,000 matching rows. Footer: **Export CSV…** (save dialog, default `okremote-audit.csv`; columns `at, actor, action, target_id, detail`, with the detail as JSON; toast `Exported N entries`) and **Close**.

15.10 About

Title "About okRemote": `okRemote <version> (<platform>)`, the line "Codename build. Connection manager for SSH, RDP, VNC and web sessions.", the settings folder path in a monospace field, and a `Portable mode` badge when applicable. Button **Close**.

15.11 Context menus

A single shared component. It opens at the pointer, clamped so it stays inside the window (at most 240 px from the right edge, and above the bottom edge by 30 px per item plus 16 px), minimum 200 px wide, 8 px radius, with 4 px padding. Items are buttons with a label and an optional key hint on the right; separators are thin lines; disabled items are dimmed; danger items use the danger color. Opening focuses the first enabled item. Hovering an item focuses it; arrow keys move focus and switch on a keyboard highlight that hover turns off. Escape, a click elsewhere or choosing an item closes it. Menus exist for: the application menu (5.4), tree rows (6.8) and tabs (12.5).

15.12 Toasts

Short messages stacked at the bottom right (14 px from the right, 34 px from the bottom, at most 440 px wide), newest at the bottom, each with a 4 px accent left border. Kinds: info, success, error. They disappear after 5 seconds unless a call asks for longer (errors usually 8 to 10 seconds, success 2 to 7); clicking one dismisses it. The region is announced politely to screen readers.

16. Settings and keyboard shortcuts

16.1 Settings storage

Settings are one JSON object in `settings.json` in the config folder (section 4.7), loaded at startup (a missing, unreadable or non-object file counts as empty) and merged over the defaults below. Any change in the interface updates memory at once and is written to disk after 400 ms without further changes, atomically (write a temporary file, then rename). The terminal block is merged field by field over its own defaults.

Key	Default	Meaning
theme	`system`	`system`, `light`, `dark`
accent	`#9fcdba`	CSS hex. Two older defaults (`#3b82f6`, `#d97757`) are rewritten to the current default at startup
density	`comfortable`	`comfortable` or `compact` (row height 28 or 22 px)
ui_scale	1	0.8 to 2; multiplies the 13 px base font size
startup_mode	`last`	`last`, `pinned`, `chooser`
pinned_workspace	null	File path of the pinned workspace
recent_workspaces	[]	Up to 12 file paths, newest first
last_open_workspaces	[]	Up to 12 file paths of vaults open at last exit
auto_lock_minutes	15	0 turns idle lock off; maximum 1440
lock_on_sleep	true	
confirm_tab_close	true	
backup_retention	10	0 to 200 backups kept per vault; 0 turns backups off
clipboard_clear_seconds	30	0 to 600; 0 never clears
terminal	see 8.6	Default terminal appearance
custom_schemes	[]	Imported color schemes (section 14.5)
keybindings	{}	Action name to combo, only where changed
sidebar_width	290	200 to 600
sidebar_collapsed	false	
recents	[]	Up to 25 `{ws, id, at}`
last_tabs	[]	Tabs open at last change, `{ws, id, quick}`
restore_tabs	`ask`	`ask`, `always`, `never`

Key	Default	Meaning
expanded	[]	Row keys of expanded folders and collapsed-workspace markers

16.2 Settings dialog

Title "Settings", extra wide, a vertical tab list on the left and the page on the right, one footer button **Done**. Changes apply immediately; there is no Save.

General. *On startup* select ("Reopen the workspaces that were open last time", "Open the pinned workspace only", "Show the workspace chooser"); *Pinned workspace* select (None, then every open workspace path and recent path; hint "Always opens first and is listed at the top of the tree."); *Restore tabs from the last session* select ("Offer on the welcome screen", "Always", "Never"); checkbox "Ask before closing a connected tab".

Appearance. *Theme* (Follow the system, Light, Dark); *Accent color* (a color picker); *Density* (Comfortable; "Compact (more rows in the tree)"); *Interface scale: N%* slider 0.8 to 2 in steps of 0.05 (hint "On top of the operating system's scale factor. Terminals keep their own font size.").

Security. *Lock after inactivity (minutes, 0 = never)*; checkbox "Lock when the computer sleeps"; *Clear copied passwords after (seconds)*; *Automatic vault backups to keep* (hint "A backup is taken each time a vault is unlocked."); then a **Workspaces** heading with one card per open workspace: the name, and buttons **Forget on this device** (only when the vault key is cached in the keychain; toast "Forgotten on this device"), **Change passphrase** (disabled while locked; opens two hidden fields *Current passphrase* and *New passphrase* and a **Change** button; the new one needs at least 8 characters, "Use at least 8 characters"; toast "Passphrase changed") and **Back up now** (disabled while locked; toast `Backup saved in <folder>` for 7 seconds). The page ends with "Telemetry: none. okRemote sends nothing anywhere."

Terminal. A note, then the terminal editor (8.6) bound to the global defaults (changes re-apply to every open terminal), and a button **Workspace terminal profiles…**.

Keyboard. A table of actions: label, the current shortcut in a key box, and buttons **Change** (the key box reads "Press keys…"; the next combination is stored; Escape cancels; a lone modifier key is ignored) and **Reset** (shown only for overridden actions).

16.3 Auto-lock

Activity is any key press, mouse press, mouse move or wheel in the window. A timer runs every 15 seconds. If the gap since the previous tick exceeds 60 seconds the computer is assumed to have slept, and when `lock_on_sleep` is on every workspace locks. Otherwise, when `auto_lock_minutes` is above 0 and no activity has happened for that long, every workspace locks. Locking closes the editor drawer and toasts "All workspaces locked". Running sessions and tunnels are not affected. The timer does nothing while no workspace is unlocked.

16.4 Shortcuts

A combo is written `Mod+Shift+K`: `Mod` is Ctrl on Windows and Linux and Cmd on macOS; `Ctrl` always means the Control key (off macOS it is the same as `Mod`); other parts are `Alt`, `Shift`, `Meta` and a key (single characters upper-cased; `+` and `=` are one key, as are `_` and `-`). The key hint shows `⌘` for `Mod` on macOS.

Action	Default	Effect
palette	Mod+K	Open the command palette
quickConnect	Mod+L	Focus the quick-connect field
filterTree	Mod+F	Show the sidebar if hidden and focus the filter
newConnection	Mod+N	New connection
newFolder	Mod+Shift+N	New folder
editSelected	F2	Edit the focused item (not while typing in a field)
closeTab	Mod+W	Close the active tab
nextTab	Ctrl+Tab	Next tab in the active pane
prevTab	Ctrl+Shift+Tab	Previous tab
splitRight	Mod+D	Split the active pane side by side
splitDown	Mod+Shift+D	Split top and bottom
lockAll	Mod+Shift+L	Lock all workspaces

Action	Default	Effect
toggleSidebar	Mod+B	Show or hide the sidebar
zoomIn, zoomOut, zoomReset	Mod+=, Mod+-, Mod+0	Terminal zoom, only when focus is inside a terminal pane

The handler listens at the top of the window in the capture phase, so these work even when a terminal has focus; it is inactive while any dialog or the palette is open. Fixed keys that cannot be rebound: Enter, Delete, arrow keys, Home, End and Alt+Up or Down in the tree; Escape in dialogs, menus, the drawer and the search bar; Ctrl or Cmd+S in the editor; Ctrl or Cmd+Shift+F, Ctrl+Shift+C and V (Cmd+C and V on macOS) in a terminal.

17. Security behaviors and the audit log

17.1 Rules the design keeps

1. Credentials exist in the clear only in memory, only while needed, and are zeroized when dropped (vault key, secrets, passwords in transit, passphrases).
2. The interface never receives a stored secret unless the user reveals or copies it; the editor learns only which fields exist and where each comes from.
3. A password is never placed on a command line or in a log. It reaches other programs only through the channels in 17.3.
4. A changed server identity (SSH host key or RDP TLS key) blocks the connection; the user must remove the old key deliberately.
5. Weak options are off by default and warn when switched on (SHA-1 `ssh-rsa`, session logging, OSC 52 clipboard writes, binding forwards beyond loopback).
6. There is no telemetry and no network traffic other than the connections the user opens (and one `curl --head` probe of the user's own HTTPS target, 10.4).
7. Remote web content runs in windows with no access to the application's command interface.
8. Imported files are parsed defensively: size limit 64 MiB, XML and CSV by safe parsers, a malformed value never aborts the whole import.

17.2 Clipboard handling

Whenever the app puts a credential on the clipboard (the copy buttons and menu items, and the launch fallbacks in sections 9 and 10), it remembers the value, waits `clipboard_clear_seconds` (default 30, 0 means never), and clears the clipboard only if it still holds exactly that value. The copy actions available to the user are **Copy user name** and **Copy password** in the tree menu, the copy buttons beside user name and password in the editor, and the clipboard fallback of the launchers. The core resolves the value as a connection would see it (inheritance applied) and records `secret.copy` with the field name only. Allowed fields: user name, password, domain, private key. Missing field: "no <field> is set". The interface toasts `Copied; the clipboard clears in N s`.

Reveal (the eye button, editor only) fetches the record's own stored secret fields (not inherited ones) and shows them in the field; it records `secret.reveal` with the owner kind.

17.3 How each client receives the password

Client	Channel	If unavailable
Embedded SSH	In memory to the SSH library	n/a
Embedded RDP	In memory to the RDP library	n/a
mstsc (Windows)	A short-lived Credential Manager entry for `TERMSRV/<host>`, removed after 30 s; an existing saved entry is never overwritten	Clipboard copy
FreeRDP	Its standard input (`/from-stdin:force`)	Clipboard copy when no user name
Windows App (macOS)	None	Clipboard copy
TigerVNC-compatible viewer	A mode-0600 obfuscated password file, deleted after 30 s	RealVNC viewers: clipboard copy
Screen Sharing (macOS)	None	Clipboard copy

Client	Channel	If unavailable
External tool	`OKREMOTE_PASSWORD` environment variable (default), or standard input, or nothing	n/a; `%PASSWORD%` on the command line is refused
Web window	None; the password is copied to the clipboard when both user name and password are set	n/a

17.4 Locking

Locking a workspace drops its vault key and empties the interface's copy of its tree; every secret and write operation then fails with "the workspace is locked". Open sessions, tunnels and transports keep running; a jump transport keeps its hop credentials in memory so it can reconnect. Unlocking is by passphrase or, when remembered, silently at open time from the OS keychain.

17.5 Audit log

Every write to a writable vault appends a row to `audit_log` (id UUIDv7, UTC time to the second, actor, action, target id, JSON detail). The actor is the operating-system user name (`USER` or `USERNAME`, else `local`). Nothing is recorded for a read-only vault. Secret values never appear in any detail.

Action	Target	Detail
workspace.create	workspace id	name
workspace.passphrase	workspace id	empty
folder.create, folder.edit	folder id	the record before (edit only) and after
folder.delete	folder id	the record before
connection.create, connection.edit	connection id	the record before (edit only) and after, including options, tags and notes
connection.delete	connection id	the record before

Action	Target	Detail
secret.change	record id	owner kind, and whether it was cleared
secret.reveal	record id	owner kind
secret.copy	record id	field name
tunnel.save	profile id	the profile after
tunnel.delete	profile id	empty
profile.save	profile id	name
profile.delete	profile id	empty
hostkey.trust	`host:port`	key type and fingerprint
hostkey.remove	`host:port`	key type
import	workspace id	source name, folder count, connection count

The audit dialog (15.9) reads the newest entries first (ordered by time then id, descending) and exports to CSV.

18. Workflows end to end

18.1 First launch

The app starts, loads settings, opens the workspaces the startup mode names (and any named on the command line), and shows the first-run screen if none is open. The screen replaces the session area (the sidebar and top bar stay): a centered card headed "Welcome to okRemote" with the text "Your connections live in a workspace. Start with a personal vault file: a single encrypted file you can keep in a synced folder or on a USB drive." and four large buttons, each an icon, a bold title and a small line:

Button	Small line	Action
Create a vault file (primary)	Set a passphrase; the app opens with an empty tree.	Opens the Vault files dialog on Create new

Button	Small line	Action
Open an existing vault	A .okvault file from another machine or a backup.	Opens the Vault files dialog on Open
Import an existing connection file	Create a vault, then import your XML connection tree into it.	Opens the create dialog and shows the toast "Create a vault first, then use Import to bring in your existing connection file" for 8 seconds
Connect to a team database (disabled)	PostgreSQL workspaces arrive in a later release.	Tooltip "Team databases arrive in a later release"

If startup fails for any reason the interface still appears, shows the first-run screen and toasts `Startup problem: <message>` for 10 seconds.

18.2 Save a connection and connect

1. Press Mod+N (or click **+ Connection**). The drawer opens on General.
2. Type a name, choose SSH, type the host. Leave the port empty.
3. Open Credentials, type a user name and password (they are encrypted on save).
4. Click **Save and connect**. The drawer closes, the new item is selected in the tree, and a tab opens in the active pane.
5. If the server is unknown, the Unknown host key dialog appears; **Trust and connect** saves the key in the workspace.

18.3 Share credentials through a folder

1. Right-click a folder, choose Edit. On General set a color.
2. On Credentials set the user name and password once.
3. Move connections into the folder (drag, or Alt+Up and Down within it). Their credential fields now read `↳ <folder>`; clear their own values with the reset icon to inherit.
4. Any option on the other sections (for example a terminal profile or a tunnel profile) set on the folder becomes the default for everything below it.

18.4 Reach a host through a jump host

1. Create SSH connections for each hop (they keep their own credentials).

2. Menu > Tunnels… > **New profile**: name it, add the hops in order, optionally add forwards, Save.
3. Edit the target connection or its folder, section Tunnel, choose the profile.
4. Connect. SSH sessions run through the chain; RDP, VNC and tool launches go through an ephemeral local forward; web connections use a SOCKS proxy. The tab shows a tunnel icon and its route in the tooltip, and the status bar names the route.

18.5 Daily use

- Mod+K, type part of a name, Enter. Mod+L, type `user@host`, Enter for a one-off session.
- Pin the connections used most (menu, or the editor checkbox); they appear in the sidebar strip and on the welcome view.
- Split a pane to watch two sessions at once; drag tabs between panes.
- Close the app with tabs open; next time the welcome view offers to restore them (or does it by itself if so configured).

18.6 Move from another tool

Import the existing file (OpenSSH config, PuTTY sessions, an RDCMan file, a CSV or an XML connection tree), read the warnings (items that could not be mapped), choose the destination and optionally a wrapping folder, and import. Credentials that could be decrypted are stored encrypted in the vault.

18.7 Lock and unlock

Mod+Shift+L, the lock button, idle timeout and sleep all lock every workspace: trees disappear, workspace rows show a lock and an **Unlock** link. Unlock by passphrase, optionally ticking "Remember on this device" so the next start opens it silently. Settings > Security shows **Forget on this device**.

18.8 A server key changed

The connection fails with the **Host key changed: connection blocked** dialog showing both fingerprints. After verifying the new key out of band, click **Open Host keys**, select the right store, **Remove** the old row, and connect again; the Unknown host key dialog then appears and the new key can be trusted.

18.9 A connection drops

The tab overlay says Disconnected (or Connection failed with the reason) with **Reconnect** and **Close tab**. Reconnect opens a fresh session in the same tab; terminals print a `[reconnected at …]` line. For tunneled sessions the shared transport reconnects by itself first (up to 10 tries, section 11.5); the tab dot turns amber while it does.

18.10 Backups and recovery

A backup is written each time a vault is unlocked (section 4.6) to `<config>/backups/<workspace id>/`. There is no restore button: to recover, close the vault, copy a backup file over the vault file (or open the backup directly through Open) and unlock it with its passphrase. A schema upgrade first leaves a `pre-migration` copy next to the vault. Changing the passphrase does not change old backups' passphrase.

19. Build, packaging and continuous integration

19.1 Toolchain

Rust stable (minimum 1.85), Node.js 20 or newer (CI uses 22), npm. Interface: React 18.3, TypeScript 5.6 in strict mode (`noUnusedLocals` , `noUnusedParameters` , `noFallthroughCasesInSwitch` ; target ES2021, bundler module resolution, `react-jsx`), Vite 5 with the React plugin (dev server on port 1420, strict port; build targets ES2022, Chrome 105, Safari 15; esbuild minify, no source maps). Terminal packages: xterm 5.5 with the fit, search, unicode11, web-links and webgl add-ons. Tauri packages: API 2.1, dialog, opener and clipboard-manager plugins, CLI 2.1. System libraries on Linux: WebKitGTK 4.1, libxdo, OpenSSL, AppIndicator, librsvg, libdbus. Windows: Visual Studio 2022 build tools with the C++ workload and WebView2.

19.2 Everyday commands

Task	Command
Run with hot reload	`cd app && npm install && npx tauri dev`
Interface only in a browser	`cd app && npm run dev` , then open `http://localhost:1420` (mock backend)
Type check	`cd app && npx tsc --noEmit`

Task	Command
Interface build	`cd app && npm run build` (type check, then Vite build into `app/dist`)
Rust tests	`cargo test --workspace`
SSH integration tests	`sudo scripts/test-ssh-server.sh`, then `OKR_SSH_HOST=127.0.0.1 OKR_SSH_PORT=2222 OKR_SSH_USER=okr OKR_SSH_PASS=okr-test cargo test -p ok-tunnel --test integration -- --ignored --test-threads=1`
Debug logging	environment variable `OKREMOTE_LOG=debug` (default `warn`)

The throwaway SSH server script (run as root on a test box) creates user `okr` with password `okr-test`, writes a drop-in config enabling password and keyboard-interactive authentication and TCP forwarding on ports 22 and 2222, generates host keys and starts or signals `sshd`. Do not run it on a machine that must not accept password logins.

19.3 Packaging

The Tauri bundle configuration: product name `okRemote`, identifier `com.halifaxsystems.okremote`, version 0.1.0, category Developer Tool, publisher and copyright strings, a short and a long description, icons (PNG sizes, `.icns`, `.ico`), and a file association for the extension `okvault` (role Editor, MIME `application/x-okremote-vault`). Windows installs the WebView2 bootstrapper when missing; macOS minimum is 12.0 with ad-hoc signing (`-`) and an `Info.plist` entry `NSLocalNetworkUsageDescription` ("okRemote connects to the SSH, RDP, VNC and web servers on your network that you add to your connections."); Debian packages use section `net`, RPM release 1.

Build scripts collect results in `dist/<version>/<platform>/`:

- `scripts/build-linux.sh`: runs the Tauri build with `APPIMAGE_EXTRACT_AND_RUN=1` and `WEBKIT_DISABLE_DMABUF_RENDERER=1`; output AppImage, `.deb`, `.rpm`.
- `scripts/build-macos.sh`: strips MacPorts and Homebrew from `PATH` so the binary links only system libraries; default is a universal build (`aarch64` and `x86_64` targets), `--native` builds one architecture; output a `.dmg` and a zipped `.app`. Signing and

notarization use `APPLE_SIGNING_IDENTITY`, `APPLE_ID`, `APPLE_PASSWORD`, `APPLE_TEAM_ID` when set.

- `scripts/build-windows.ps1`: output an MSI, an NSIS setup program, and a portable zip containing `okRemote.exe` plus a `portable` marker file (settings then live in `data/` beside the executable). The output folder is emptied first so file name capitalization is not inherited from older builds.

`CARGO_TARGET_DIR` can move the Rust build output out of a synchronized folder.

19.4 Continuous integration

One workflow named `ci` runs on pull requests, pushes to `main`, `v*` tags and manual dispatch.

Job	Runs on	Steps
check (matrix, no fail-fast)	ubuntu-24.04, windows-latest, macos-14	Checkout; Rust stable with clippy and rustfmt; Node 22 with npm cache keyed on `app/package-lock.json`; on Linux install the system packages; in `app`: `npm ci`, `npx tsc --noEmit`, `npx vite build`; `cargo test --workspace`; `cargo clippy --workspace -- -D warnings` (allowed to fail)
integration	ubuntu-24.04	Start the throwaway SSH server; run the ignored `ok-tunnel` integration tests against it
package	windows-latest, ubuntu-22.04, macos-14 (universal target)	Only on `v*` tags or manual dispatch, after `check`; `npm ci` and `npx tauri build`; upload installers (`msi`, `exe`, `dmg`, `AppImage`, `deb`, `rpm`) as the artifact `okremote-<OS>`. Installers are unsigned

Unit tests number about 55 (model, crypto, store, importers, launchers, tunnel parsing, shell helpers); the integration suite covers a shell round trip with a window change, a wrong password, a two-hop jump chain, local and SOCKS5 forwards, refusing an exposed

bind without confirmation and transport reuse.

20. Appendices

Appendix A. Commands

Arguments are named and sent as JSON; `ws` is a workspace id, `owner` is `folder` or `connection`, `target` in the session commands is `{ws, conn_id, quick}` (a saved connection, or a quick target as in 3.10).

Command	Arguments	Returns
app_info	none	product, version, platform, portable, config_dir, logs_dir, vault_ext
settings_get / settings_save	none / settings object	object / nothing (refuses a non-object)
startup_requests	none	paths requested at launch (consumed once)
default_vault_path	none	`<Documents>/OkRemote/personal.okvault`
quick_parse	input	quick target (3.10)
prompt_respond	id, value	nothing
color_scheme_import	path	scheme `{name, colors}`
ws_list	none	workspace infos sorted by name
ws_create	path, name, passphrase, remember	workspace info
ws_open	path, passphrase?, remember?	workspace info (locked if no key was available)
ws_unlock	id, passphrase, remember	workspace info
ws_lock, ws_lock_all, ws_close	id / none / id	info / nothing / nothing
ws_snapshot	id	snapshot

Command	Arguments	Returns
ws_rename	id, name	workspace info
ws_change_passphrase	id, old, new	nothing
ws_forget_device, ws_remembered	id	nothing, boolean
ws_backup_now	id	backup folder path
folder_save, conn_save	ws, folder or conn	the saved record
folder_delete, conn_delete	ws, id	nothing
folder_duplicate, conn_duplicate	ws, id, target? (absent: same parent; empty text: top level; else folder id)	the copy
items_move	ws, items[{kind, id}], parent, before	nothing
conns_set_option	ws, ids, key, value (null removes)	count changed
secret_get	ws, owner, id	the record's own secret (audited)
secret_set	ws, owner, id, secret or null	nothing
secret_patch	ws, owner, id, patch (field to string or null)	nothing
secret_summary	ws, owner, id	per field: own, inherited_from, value (non-secret fields only)
secret_copy	ws, owner, id, field	seconds until the clipboard clears

Command	Arguments	Returns
tunnel_save, tunnel_delete	ws, tunnel / id	saved profile / nothing
tunnel_start	ws, id, allowExposed	running forwards
tunnel_stop	ws, id	nothing
forward_stop	id	nothing
tunnels_status	none	forwards and transports
profile_save, profile_delete	ws, profile / id	saved profile / nothing
hostkeys_list	ws or null	host keys
hostkey_delete	ws or null, host, port, keyType	nothing
audit_list	ws, limit? (default 1000)	entries newest first
audit_export_csv	ws, path	count written
import_preview	kind, path or null, password or null	preview (14.1)
import_commit	ws, token, parent, wrap	`{folders, connections, tunnels}`
export_json	ws, path	connection count
session_open	target, cols, rows, binary channel	session info `{id, title, route, terminal_profile, osc52, fixed_cols, fixed_rows, log_file}`
session_write, session_resize, session_close	id with data / cols and rows / nothing	nothing
sessions_list	none	`{id, title, ws, conn_id}` for live SSH sessions (not used by the interface)

Command	Arguments	Returns
rdp_open	target, width, height, binary channel	`{id, title, route, width, height}`
rdp_input, rdp_close	id with input / id	nothing
launch_external	target	`{client, route, password_copied, message}`
web_open	target	same shape

Appendix B. Error codes

Every failure is `{code, message, detail?}`.

Code	Meaning and interface behavior
locked	The workspace is locked. The interface refreshes the workspace list
read_only	Vault written by a newer version, opened read-only
wrong_passphrase	Shown inside the unlock dialog
not_a_vault, too_new, not_found, invalid, storage	Shown as an error toast with the message
conflict	Stale record; `detail` is the current record. Opens the conflict dialog (7.6)
password_required, wrong_password	Import file password flow (14.1)
import	Any other import error
host_key	The server key was not trusted
auth	Authentication failed
cancelled	The user dismissed a prompt. Shown as an info toast or as "Sign-in was cancelled." in a terminal tab

Code	Meaning and interface behavior
ssh	Other SSH errors, with the message (for example "could not reach host:22: …", "timed out connecting to …")
rdp	Embedded RDP errors (9.2)
external	The connection must use an external RDP client
launch	An external client could not start
tunnel	A forward could not start
keychain	The OS keychain refused to store the key
io, error	File errors and everything else

Appendix C. Built-in terminal color schemes

Each line: name, background, foreground, then the 16 ANSI colors black through white, then bright black through bright white. Cursor color equals the foreground unless noted; selection color is `#3b4b6b` (dark schemes) or `#b4d5fe` (light) unless noted.

```
OkRemote Dark   bg #181818 fg #dcdcdc
  #2a2a2a #f0717a #8bd17c #e8c46a #6aa7f8 #c792ea #5fd4d8 #dcdcdc
  #6b6b6b #ff8f97 #a6e59a #ffd98a #8fbfff #dcb0ff #86eaee #ffffff
OkRemote Light  bg #faf9f7 fg #1f1e1d
  #1f2937 #c62828 #2e7d32 #9a6b00 #1d4ed8 #8e24aa #00838f #e5e7eb
  #6b7280 #e53935 #43a047 #b58900 #2563eb #ab47bc #0097a7 #ffffff
Solarized Dark  bg #002b36 fg #839496
  #073642 #dc322f #859900 #b58900 #268bd2 #d33682 #2aa198 #eee8d5
  #586e75 #cb4b16 #586e75 #657b83 #839496 #6c71c4 #93a1a1 #fdf6e3
Solarized Light bg #fdf6e3 fg #657b83  selection #eee8d5
  #073642 #dc322f #859900 #b58900 #268bd2 #d33682 #2aa198 #eee8d5
  #002b36 #cb4b16 #586e75 #657b83 #839496 #6c71c4 #93a1a1 #fdf6e3
Dracula         bg #282a36 fg #f8f8f2  selection #44475a
  #21222c #ff5555 #50fa7b #f1fa8c #bd93f9 #ff79c6 #8be9fd #f8f8f2
  #6272a4 #ff6e6e #69ff94 #ffffa5 #d6acff #ff92df #a4ffff #ffffff
Nord            bg #2e3440 fg #d8dee9  selection #434c5e
  #3b4252 #bf616a #a3be8c #ebcb8b #81a1c1 #b48ead #88c0d0 #e5e9f0
  #4c566a #bf616a #a3be8c #ebcb8b #81a1c1 #b48ead #8fbcbb #eceff4
Gruvbox Dark    bg #282828 fg #ebdbb2  selection #504945
```

```
  #282828 #cc241d #98971a #d79921 #458588 #b16286 #689d6a #a89984
  #928374 #fb4934 #b8bb26 #fabd2f #83a598 #d3869b #8ec07c #ebdbb2
Campbell        bg #0c0c0c fg #cccccc
  #0c0c0c #c50f1f #13a10e #c19c00 #0037da #881798 #3a96dd #cccccc
  #767676 #e74856 #16c60c #f9f1a5 #3b78ff #b4009e #61d6d6 #f2f2f2
One Light       bg #fafafa fg #383a42  selection #e5e5e6
  #383a42 #e45649 #50a14f #c18401 #0184bc #a626a4 #0997b3 #fafafa
  #4f525e #e06c75 #98c379 #e5c07b #61afef #c678dd #56b6c2 #ffffff
```

The picker groups schemes under Dark and Light. Darkness of a built-in scheme is fixed; for an imported one it is computed (14.5). An unknown scheme name falls back to OkRemote Dark.

Appendix D. Icons

Every icon is drawn from one SVG path on a 24 by 24 grid, stroked (no fill) at width 1.8 with round caps and joins in the current text color; default size 16 px. Unknown names draw the three-dot icon. A connection of protocol `https` uses the `http` icon.

```
folder      M3 6.5A1.5 1.5 0 0 1 4.5 5H9l2 2h8.5A1.5 1.5 0 0 1 21 8.5v9a1.5
1.5 0 0 1-1.5 1.5h-15A1.5 1.5 0 0 1 3 17.5z
folderOpen  M3 6.5A1.5 1.5 0 0 1 4.5 5H9l2 2h8.5A1.5 1.5 0 0 1 21
8.5V10H6.2a1.5 1.5 0 0 0-1.4 1L3 17zM3 17l2.2-5.6A1.5 1.5 0 0 1 6.6 10.5H22l-
2.4 7.1a1.5 1.5 0 0 1-1.4.9H4.5
ssh         M4 5h16v14H4zM7 9l3 3-3 3M12 15h5
terminal    M4 5h16v14H4zM7 9l3 3-3 3M12 15h5
rdp         M3 4h18v12H3zM8 20h8M12 16v4
vnc         M3 4h18v12H3zM8 20h8M12 16v4M8 8h8v4H8z
http        M12 3a9 9 0 1 0 0 18 9 9 0 0 0 0-18M3 12h18M12 3c2.5 2.7 3.5 5.7
3.5 9s-1 6.3-3.5 9c-2.5-2.7-3.5-5.7-3.5-9S9.5 5.7 12 3
external    M14 4h6v6M20 4l-9 9M18 14v5a1 1 0 0 1-1 1H5a1 1 0 0 1-1-1V7a1 1 0
0 1 1-1h5
lock        M6 11h12v9H6zM8.5 11V8a3.5 3.5 0 0 1 7 0v3
unlock      M6 11h12v9H6zM8.5 11V8a3.5 3.5 0 0 1 6.7-1.4
vault       M4 4h16v16H4zM12 9a3 3 0 1 0 0 6 3 3 0 0 0 0-6M12 9V7M12 17v-2M9
12H7M17 12h-2
star        M12 3.5l2.6 5.3 5.9.9-4.3 4.1 1 5.8L12 16.9l-5.2 2.7 1-5.8-4.3-
4.1 5.9-.9z
plus        M12 5v14M5 12h14
close       M6 6l12 12M18 6L6 18
search      M11 4a7 7 0 1 0 0 14 7 7 0 0 0 0-14M20 20l-4-4
tunnel      M3 18V11a9 9 0 0 1 18 0v7M7 18v-6a5 5 0 0 1 10 0v6M3 18h4M17 18h4
```

```
chevron     M9 6l6 6-6 6
menu        M4 7h16M4 12h16M4 17h16
split       M4 4h16v16H4zM12 4v16
splitDown   M4 4h16v16H4zM4 12h16
sidebar     M4 4h16v16H4zM9 4v16
key         M15 7a4 4 0 1 0-3.9 4.9L4 19v2h3v-2h2v-2h2l1.1-1.1A4 4 0 0 0 15
7M16 6.5h.01
history     M3 12a9 9 0 1 0 3-6.7M3 4v4h4M12 7v5l3 2
importIcon  M12 3v12M7 10l5 5 5-5M5 21h14
copy        M8 8h11v13H8zM5 16H4V3h12v1
eye         M2 12s3.5-7 10-7 10 7 10 7-3.5 7-10 7S2 12 2 12M12 9a3 3 0 1 0 0
6 3 3 0 0 0 0-6
reset       M4 4v6h6M4.5 15a8 8 0 1 0 1.9-8.3L4 10
warn        M12 3l10 18H2zM12 10v5M12 18h.01
dots        M5 12h.01M12 12h.01M19 12h.01
refresh     M20 11a8 8 0 0 0-14.9-3M4 4v4h4M4 13a8 8 0 0 0 14.9 3M20 20v-4h-4
gear        M12 9a3 3 0 1 0 0 6 3 3 0 0 0 0-6M19.4 15a1.7 1.7 0 0 0 .3
1.8l.1.1a2 2 0 1 1-2.8 2.8l-.1-.1a1.7 1.7 0 0 0-1.8-.3 1.7 1.7 0 0 0-1
1.5V21a2 2 0 1 1-4 0v-.1a1.7 1.7 0 0 0-1.1-1.5 1.7 1.7 0 0 0-1.8.3l-.1.1a2 2
0 1 1-2.8-2.8l.1-.1a1.7 1.7 0 0 0 .3-1.8 1.7 1.7 0 0 0-1.5-1H3a2 2 0 1 1 0-
4h.1a1.7 1.7 0 0 0 1.5-1.1 1.7 1.7 0 0 0-.3-1.8l-.1-.1a2 2 0 1 1 2.8-
2.8l.1.1a1.7 1.7 0 0 0 1.8.3H9a1.7 1.7 0 0 0 1-1.5V3a2 2 0 1 1 4 0v.1a1.7 1.7
0 0 0 1 1.5 1.7 1.7 0 0 0 1.8-.3l.1-.1a2 2 0 1 1 2.8 2.8l-.1.1a1.7 1.7 0 0
0-.3 1.8V9a1.7 1.7 0 0 0 1.5 1H21a2 2 0 1 1 0 4h-.1a1.7 1.7 0 0 0-1.5 1z
```

The application icon is a 2048 by 2048 master (neutral grey surface with a seafoam accent) exported to 512, 256, 128, 64, 32 and the Windows and macOS icon containers and the Store logo sizes.